Learning-to-Anneal for Combinatorial Optimization Problems on Graphs

***Abstract*—Many graph-based combinatorial optimization (GCO) problems are NP-hard, posing significant challenges in finding high-quality solutions efficiently. Applications such as traffic scheduling, logistics routing, and virtual-machine placement often involve families of related graph instances, making them natural scenarios for learning over graph distributions. In this work, we propose a Learning-to-Anneal (L2A) framework for population-based GCO problems. It learns a stochastic transition policy over the complete solution space and recursively refines candidate solutions. The policy is pretrained over a graph distribution and then applied to newly sampled graph instances with low inference latency. Central to L2A is a random masking training scheme and the corresponding Transformer-based policy network. The random masking gradually increases the number of activated graph nodes during training, which imposes a curriculum learning (CL) strategy on the attention modeling in the Transformer networks. This facilitates a more stable and scalable graph feature learning. To further improve the training convergence, another filter function-based curriculum is proposed to guide training from a smoother effective objective toward the original objective. Experimental results on Max-Cut problems show that L2A achieves high-quality solutions over multiple graph distributions with low inference latency after pretraining. The proposed method outperforms strong learning-based baselines and is competitive with or better than time-limited Gurobi in several large-scale settings. It also generalizes well across graph sizes.**

I. Introduction

Graph combinatorial optimization (GCO) problems arise in a wide range of applications, including social networks, transportation, telecommunications, scheduling, routing, and resource allocation. Representative examples include Max-Cut [4], graph coloring [8], and traveling salesman problems [32]. Typically, GCO problems are NP-hard, and their decision versions are NP-complete [8], [28]. A large body of classical optimization methods has been developed for GCO problems. Exact algorithms, such as cutting-plane and branch-and-bound methods, can provide near optimal solutions, but their computational cost is often prohibitive for large instances [30], [9]. In contrast, heuristic solvers are usually faster, but their performance often depends on problem-specific designs and lacks theoretical guarantees [12], [24], [25]. Overall, classical methods rely on explicit search procedures or hand-designed rules, making them computationally expensive or difficult to adapt across problem structures. These limitations have motivated machine learning (ML) methods that learn instance-specific search behavior or solution patterns in a data-driven manner. Existing *instance-level* approaches include supervised learning schemes [42], [43], [29], unsupervised learning schemes [34], [44], [31], and sampling-based learning schemes [19], [38], [10]. However, these methods solve similar instances again and again, which can be inefficient when related optimization tasks appear frequently in real systems. For example, an UberEats delivery platform in New York City repeatedly optimizes routing over a road network with regular traffic, demand, and connectivity patterns. Although each instance may differ in edge weights, requests, or local constraints, these variations occur around a shared urban structure. Instance-level methods solve each instance from scratch and therefore fail to exploit this distributional regularity. This observation motivates a *population-based setting*: related instances are viewed as samples from the same underlying graph distribution, over which a solver is pretrained and then applied to newly sampled instances with fast inference.

A. Population-based Learning Methods

Population-based learning methods, also known as *amortized optimization* methods [3], train solvers over a distribution of graph instances. Unlike instance-based learning methods, they shift part of the computational burden from online optimization to offline training, enabling the pretrained model to solve unseen instances from the same or related distributions with substantially lower inference cost. Representative population-based approaches include supervised learning, unsupervised learning, and reinforcement learning schemes. For *supervised schemes*, solver-generated solutions are used as labels to train neural models over graph distributions. Representative works include distribution-level training with test-time refinement [27] and Transformer-based neural combinatorial optimization [13]. For unsupervised schemes, neural solvers are trained over graph distributions without exact solution labels, such as probabilistic-loss-based GNNs [21] and diffusion-based graph solvers [39]. These methods avoid labeled data, but their surrogate objectives may not align well with the final solution quality. Reinforcement learning (RL) methods offer an alternative by training sequential policies with objective-derived rewards. [22] formulated greedy graph optimization as a generic MDP and combined graph embeddings with Q-learning. [6] used pointer networks to learn constructive policies from sampled problem instances, while [33] trained neural samplers over graph distributions through variational annealing. [46] applied GFlowNets to distribution-level GCO problems, learning generative policies for sampling high-quality solutions across graph instances. These works demonstrate the potential of population-based RL methods for GCO problems.

B. Challenges and Basic Idea

Despite the progress of population-based learning, existing schemes still face several challenges. LSTM-based schemes, such as the learning-to-cut method [41], learn sequential decision rules over problem instances, but the compact recurrent

states limit its scalability on large graphs. GNN-based schemes, including S2V-DQN [22], Erdős probabilistic GNN [21], variational annealing on graphs [33], and GFlowNets [46], train the shared policies or samplers over graph distributions. However, GNN models struggle to capture long-range dependencies and cannot outperform some commercial solvers, such as Gurobi, on large graphs. Transformer-based schemes improve global correlation modeling through attention [26], [13]. However, existing learning methods typically generate solutions via constructive decoding or learned action sequences, rather than performing stochastic transitions across full-size solutions. Moreover, population-based learning models remain difficult to train for large-scale GCO problems, where the loss landscape is highly irregular and contains many poor local optima.

These challenges motivate our *Learning-to-Anneal* (L2A) framework, which combines Transformer-based global dependency modeling, full-space transitions with annealing, and curriculum learning (CL) for population-based training. Specifically, instead of constructing a solution via one-shot prediction, L2A initializes a candidate solution and refines it by **learning a transition policy over the entire solution space**. The transition policy is pretrained over a graph distribution and then recursively applied to each newly sampled graph instance during inference. We then parameterize this transition policy with a Transformer network. Given the current solution and graph embedding, the policy network outputs a sampling distribution for the next solution. The resulting extended Transformer uses attention to implement these conditional unmasking steps and captures global correlations more directly than recurrent policies, such as LSTMs, or local message-passing policies, such as GNNs. Unrolled over multiple transition steps, the Transformer iteratively updates both the candidate solution and the graph embedding. Finally, to address the poor-convergence issue caused by the irregular loss landscape of GCO problems, we introduce the CL paradigm. The first curriculum is implemented by random masking on the graph-based attention, which enables training to start from fewer active nodes and gradually expand to the full-size graph; The second curriculum is based on filter functions, which enable training to start from a smoother effective objective and gradually reduces the filtering strength until the policy optimizes the original objective.

C. Contributions and Outline

The above discussion suggests three key features for a general-purpose population-based solver of GCO problems. First, the solver should learn the latent characteristics shared by graph instances from the same distribution, so that the common structure captured during training can be reused for fast inference on new instances. Second, to handle the exponentially large solution space, it is better for a solver to employ a recursive transition process and to learn the transition policy over the complete solution space, rather than relying only on one-shot construction or local action selections. Third, the solver should remain stable and scalable when trained on large graphs, where the loss landscape is highly irregular and contains many poor local optima. To achieve these requirements, we propose a *Learning-to-Anneal* framework for solving the GCO problems, with the transition policy parameterized by an extended Transformer and the CL paradigm applied to improve the training performance. The main contributions of this work are summarized as follows:

- We propose a population-based *Learning-to-Anneal* (L2A) framework for GCO problems. The proposed framework learns a stochastic transition policy over the complete solution space and recursively refines candidate solutions. To train this policy, we develop trajectory-based policy-gradient and actor-critic schemes over graph distributions, where the actor-critic formulation reduces gradient variance and stabilizes trajectory-level learning.
- We introduce curriculum learning strategies to improve the training convergence on large-scale GCO problems. One curriculum implements random masking on the graph-based attention, which starts from fewer active nodes and progressively expands to the full graph, facilitating a more stable and scalable graph feature learning. Another filter-function-based curriculum starts from a smoother effective objective and gradually reduces the filtering strength toward the original highly spiky objective.
- We propose a blockwise sampling method to strike a balance between sampling efficiency and model expressivity. It factorizes the transition distribution through conditional node unmasking and subsumes mean-field sampling and autoregressive sampling as two special cases. We also design an extended Transformer policy network that takes the current candidate solution and graph embedding as inputs, conditionally samples the next solution, and updates the graph embedding through unrolled transition steps.
- We evaluate the proposed framework on Max-Cut problems over multiple graph distributions and graph sizes. The results show that L2A achieves high-quality solutions with low inference latency after pretraining on graph distributions. It outperforms strong learning-based baselines, is competitive with or better than time-limited Gurobi in several large-scale settings, and generalizes well across graph sizes.

The remainder of this paper is organized as follows. Sec. II formulates the population-based GCO problem, presents the generic L2A framework, and introduces the random-masking sampling method. Sec. III provides the design of the extended Transformer policy network based on this sampling method. Sec. V introduces the proposed filter function-enhanced curriculum learning paradigm. Sec.V reports the results of ablation studies and performance evaluations.

II. Problem Formulation and L2A Framework

We first formulate population-based combinatorial optimization problems on graphs, and then present a generic learning-based approach inspired by classical simulated annealing [23], referred to as learning-to-anneal (L2A).

A. Formulation of Population-based GCO Problems

Consider a weighted graph $G = (V, E, w)$, where V and E denote the node and edge sets, respectively, and $w(i, j)$ is a

weight associated with edge (i,j). We associate each node i with a binary variable, i.e., $x_i \in \{0,1\}$, and each edge (i,j) with a binary variable, i.e., $z_{i,j} \in \{0,1\}$. The resulting node- and edge-variable vectors are $\boldsymbol{x} \in \{0,1\}^{|V|}$ and $\boldsymbol{z} \in \{0,1\}^{|E|}$, respectively. We assume that G is drawn from a distribution $\mathbb{D}$ and contains N nodes, i.e., $|V| = N$, and study the population-based GCO problems of the form:

$$\left\{ \arg\min_{\boldsymbol{x},\boldsymbol{z}} \ f(\boldsymbol{x},\boldsymbol{z},G) \mid G \in \mathcal{G}, \boldsymbol{x} \in \{0,1\}^{|V|}, \boldsymbol{z} \in \{0,1\}^{|E|} \right\}, \tag{1}$$

where $\mathcal{G} = \{G = (V,E,w) \sim \mathbb{D} \mid |V| = N\}$ denotes the set of graph instances and $f : \{0,1\}^{|V|+|E|} \times \mathcal{G} \to \mathbb{R}$ denotes the objective function. For example, in the **Max-Cut problem** [22], the objective in (1) can be specified as

$$f(\boldsymbol{x},G) = -\sum_{1 \le i < j \le N} \frac{1}{2}(1 - x_i x_j) w(i,j), \tag{2}$$

where each graph node is associated with a binary variable x_i. For the **graph-coloring problem**, we represent each color assignment by an edge between vertex i and color label ℓ, and associate it with a binary variable $z_{i,\ell}$, for $i = 1,\dots,N$ and $\ell = 1,\dots,K$. The objective in (1) can then be specified as

$$f(\boldsymbol{z},G) = \sum_{(i,j)\in E} \sum_{\ell=1}^{K} z_{i,\ell} z_{j,\ell} + \sum_{i=1}^{N} \left(\sum_{\ell=1}^{K} z_{i,\ell} - 1 \right)^2, \tag{3}$$

where the first term is associated with all edges, and the second term enforces exactly one color for each vertex. More broadly, as shown in [28], problems of the general form in (1) can be equivalently represented by Ising-type formulations, including formulations involving both node and edge variables.

In this work, we aim to rapidly solve the Max-Cut problem, i.e., the GCO formulation in (1) with objective in (2), to support real-time applications, such as vehicular routing and virtual machine placement. That is, for each graph $G \in \mathcal{G}$, a near-optimal solution $\boldsymbol{x}^*(G) = \arg\min_{\boldsymbol{x}} \ f(\boldsymbol{x};G)$ should be obtained with low computational latency. However, directly solving this NP-complete problem family is highly challenging.

To reduce the optimization difficulty, inspired by [20], [47], we consider a recursive search process consisting of T updates. Each update is governed by a probabilistic mapping $\pi_t : \{0,1\}^N \times \mathcal{G} \to \mathcal{P}^N$ as follows

$$\boldsymbol{x}^t \ \sim \ \pi_t(\cdot \mid \boldsymbol{x}^{t-1}, G), \ \text{for } t = 1,2,\cdots,T, \tag{4}$$

where $\mathcal{P}_N$ represents the set of all probability mass functions over $\{0,1\}^N$. The mapping π_t takes the graph G and the current solution $\boldsymbol{x}^{t-1}$ as inputs and returns a probability distribution over candidate solutions, from which $\boldsymbol{x}^t$ is sampled. Such a probabilistic search mechanism is particularly suitable for high-dimensional combinatorial spaces because it can escape poor local minima while maintaining exploration. The mappings $\{\pi_t\}_{t=1}^T$ are designed so that the resulting solution sequence $\{\boldsymbol{x}^t\}$ progressively approaches $\boldsymbol{x}^*(G)$.

An example of such a recursive probabilistic search process is the simulated annealing (SA) algorithm [23], [12], which introduces stochastic transitions that balance exploration and exploitation. Specifically, the annealing process is controlled by a temperature parameter $\lambda_t \in [0,1]$, which decreases over iterations to gradually reduce randomness. The new solution at step t is given by

$$\boldsymbol{x}^t = \begin{cases} \widetilde{\boldsymbol{x}}^t, & \text{with probability } \pi(\Delta_t, \lambda_t), \\ \boldsymbol{x}^{t-1}, & \text{with probability } 1 - \pi(\Delta_t, \lambda_t), \end{cases} \tag{5}$$

where $\widetilde{\boldsymbol{x}}^t$ is a neighboring binary vector of $\boldsymbol{x}^{t-1}$, obtained by randomly choosing one bit of $\boldsymbol{x}^{t-1}$ and flipping it. The transition probability is given by [23]:

$$\pi(\Delta_t, \lambda_t) = \begin{cases} 1, & \text{if } \Delta_t \le 0, \\ \exp\left(-\dfrac{\Delta_t}{\lambda_t}\right), & \text{otherwise}, \end{cases} \tag{6}$$

where $\Delta_t = f(\widetilde{\boldsymbol{x}}^t, G) - f(\boldsymbol{x}^{t-1}, G)$. When $\Delta_t \le 0$, $\widetilde{\boldsymbol{x}}^t$ improves the objective and is always accepted. When $\Delta_t > 0$, the transition is accepted with a probability controlled by the temperature λ_t, where a higher temperature increases the likelihood of accepting worse solutions $\widetilde{\boldsymbol{x}}^t$. As the temperature gradually decreases, the algorithm transitions from exploration to exploitation.

B. General Learning-to-Anneal Framework

It is well-known that simulated annealing (SA) can be prohibitively slow to converge, especially on large-scale instances. To tackle this problem, as shown in Fig. 1, we adopt a learning-to-anneal (L2A) scheme, where the annealing transition process at each step is performed by a neural network Π_θ with learnable parameters θ, i.e., $\Pi_\theta : \{0,1\}^N \times \mathcal{G} \to \mathcal{P}_N$. Let $\tau \triangleq (\boldsymbol{x}^0, \boldsymbol{x}^1, \cdots, \boldsymbol{x}^T)$ denote the solution trajectory. We then define the following trajectory-level loss function

$$J_\lambda(\theta;\tau,G) = g(\tau,G) + \lambda \ \log p(\tau \mid \theta, \boldsymbol{Z}(G)), \tag{7}$$

where the trajectory-level metric $g(\tau,G) = \sum_{t=0}^{T} f(\boldsymbol{x}^t, G)$, and $\boldsymbol{Z}(G)$ is the graph embedding. Under expectation, the term $\lambda \log p(\tau|\theta, \boldsymbol{Z}(G))$ is equivalent to $-\lambda H(p(\tau|\theta, \boldsymbol{Z}(G)))$ and encourages exploration when minimizing the loss. Furthermore, the function $g(\tau,G)$ measures the optimization performance of the sampled trajectory τ on a graph instance G, while $p(\tau|\theta, \boldsymbol{Z}(G))$ is the transition probability based on the corresponding graph embedding $\boldsymbol{Z}(G)$. Similar to (4), here the transition follows:

$$\boldsymbol{x}^t \sim \Pi_\theta(\cdot \mid \boldsymbol{x}^{t-1}, \boldsymbol{Z}(G)), \quad \text{for } t = 1,2,\dots,T. \tag{8}$$

Based on (7) and (8), we define the following expected loss over a graph distribution:

$$J_\lambda(\theta) \triangleq \mathbb{E}_{G\sim\mathbb{D}} \mathbb{E}_{\tau=(\boldsymbol{x}^0,\cdots,\boldsymbol{x}^T):\boldsymbol{x}^t\sim\Pi_\theta(\cdot|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G))} \left[J_\lambda(\theta;\tau,G) \right], \tag{9}$$

and the learning problem becomes

$$\min_\theta \ J_\lambda(\theta), \tag{10}$$

which is solved using a gradient-based method. Note that (9) contains two nested expectations. The outer expectation $\mathbb{E}_{G\sim\mathbb{D}}[\cdot]$ arises because we aim to learn a population-based solver whose policy Π_θ generalizes across the graph distribution; in practice, it is approximated by averaging the

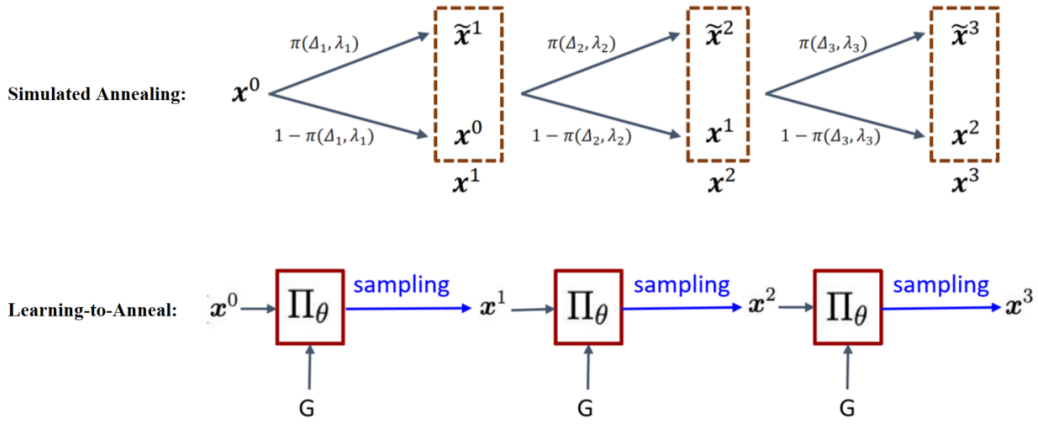


Fig. 1. Overview of the simulated annealing (top) and learning-to-anneal (bottom) frameworks. Shown here are two examples with $T = 3$ steps. At each step t of the simulated annealing framework, an annealing process controlled by a temperature parameter λ_t is applied to update the binary vector; at each step t of the learning-to-anneal framework, a neural network Π_θ is applied to the input $(\boldsymbol{x}^{t-1}, G)$ and outputs a probability distribution $\Pi_\theta(\cdot|\boldsymbol{x}^{t-1}, G)$, from which sample $\boldsymbol{x}^t$ is drawn.

loss over sampled graphs $G \sim \mathbb{D}$. The inner expectation $\mathbb{E}_{\tau\sim\Pi_\theta}[\cdot]$ accounts for the stochasticity of the learned transition policy because each trajectory is sampled from the probability distributions output by the network. For each graph instance G, this expectation is approximated by averaging over sampled trajectories $\tau = (\boldsymbol{x}^0, \boldsymbol{x}^1, \cdots, \boldsymbol{x}^T)$. Although the ultimate quantity of interest is the final objective value $f(\boldsymbol{x}^T, G)$, aggregating the objective values along the trajectory encourages progressive improvement throughout the search process. Moreover, training θ over the graph distribution $\mathbb{D}$ amortizes the search cost across graph instances, shifting the computational burden from online combinatorial optimization to offline neural-network training. At inference, for a new graph instance G, we expect the recursive application of the trained model Π_θ to produce a near-optimal solution, i.e., $\boldsymbol{x}^*(G) \approx \boldsymbol{x}^T$. Since each update only requires a feedforward mapping and a sampling operation, the resulting solver can be executed repeatedly with low latency, making real-time deployment feasible.

C. Policy-Gradient and Actor-Critic L2A Schemes

1) Policy Gradient: Substituting (7) into (9) yields the expanded form of $J_\lambda(\theta)$ in (11).

Based on (8), the probability of a T-step trajectory τ is given as follows

$$\begin{aligned} p(\tau \mid \theta, \boldsymbol{Z}(G)) &= p(\boldsymbol{x}^0) \prod_{t=1}^{T} p(\boldsymbol{x}^t \mid \boldsymbol{x}^{t-1}) \\ &= p(\boldsymbol{x}^0) \prod_{t=1}^{T} \Pi_\theta(\boldsymbol{x}^t \mid \boldsymbol{x}^{t-1}, \boldsymbol{Z}(G)). \end{aligned} \tag{12}$$

Hence, the corresponding log-probability is

$$\begin{aligned} \log p(\tau|\theta, \boldsymbol{Z}(G)) &= \log(p(\boldsymbol{x}^0)) \\ &+ \sum_{t=1}^{T} \log\Big(\Pi_\theta\big(\boldsymbol{x}^t|\boldsymbol{x}^{t-1}, \boldsymbol{Z}(G)\big)\Big), \end{aligned} \tag{13}$$

where $p(\boldsymbol{x}^0)$ denotes the distribution of the initial solution and is independent of θ, i.e., $\nabla_\theta \log(p(\boldsymbol{x}^0)) = 0$. We assume that $\Pi_\theta(\boldsymbol{x}^t \mid \boldsymbol{x}^{t-1}, \boldsymbol{Z}(G))$ is differentiable with respect to θ. Therefore, based on (11) and (13), the policy gradient of (9) is given by (14), where

$$\begin{aligned} &A_\lambda(\tau|\theta, G) = g(\tau, G) + \lambda \log p(\tau|\theta, \boldsymbol{Z}(G)) - b(G), \\ &b(G) = \mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)}\left[g(\tau, G) + \lambda \log p(\tau|\theta, \boldsymbol{Z}(G))\right]. \end{aligned} \tag{15}$$

Detailed proof of (14) and (15) is given in Appendix A.

2) Actor-Critic: Policy-gradient methods often suffer from high-variance gradient estimates, which can lead to inefficient learning [17], [48]. To reduce this variance, we introduce an actor-critic approach. For each graph instance, we define the value function $V(\boldsymbol{x}^t, G)$ as the expected cumulative return starting from $\boldsymbol{x}^t$:

$$\begin{aligned} V(\boldsymbol{x}^t, G) \triangleq \mathbb{E}_{\tau^t=(\boldsymbol{x}^{t+1},\cdots,\boldsymbol{x}^T):\boldsymbol{x}^k\sim\Pi_\theta(\cdot|\boldsymbol{x}^{k-1},\boldsymbol{Z}(G))} \\ \left[g(\tau^t, G) + \lambda \log \prod_{k=t+1}^{T} \Pi_\theta(\boldsymbol{x}^k|\boldsymbol{x}^{k-1}, \boldsymbol{Z}(G))\right], \end{aligned} \tag{16}$$

where $\tau^t = (\boldsymbol{x}^{t+1}, \cdots, \boldsymbol{x}^T)$ denotes the sampled trajectory suffix after $\boldsymbol{x}^t$, and $g(\tau^t, G)$ measures its optimization performance on graph G. Based on (16), we define the return-to-go of τ^t as

$$R(\tau^t, G, \lambda) \triangleq g(\tau^t, G) + \lambda \log \prod_{k=t+1}^{T} \Pi_\theta(\boldsymbol{x}^k|\boldsymbol{x}^{k-1}, \boldsymbol{Z}(G)). \tag{17}$$

$$J_\lambda(\theta) \triangleq \mathbb{E}_{G\sim\mathbb{D}}\mathbb{E}_{\tau=(\boldsymbol{x}^0,\cdots,\boldsymbol{x}^T):\boldsymbol{x}^t\sim\Pi_\theta(\cdot|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G))}\left[J_\lambda(\theta;\tau,G)\right] = \mathbb{E}_{G\sim\mathbb{D}}\left[\int_\tau \big(g(\tau,G)+\lambda\log p(\tau|\theta,\boldsymbol{Z}(G))\big)p(\tau|\theta,\boldsymbol{Z}(G))d\tau\right]. \tag{11}$$

$$\nabla_\theta J_\lambda(\theta) = \mathbb{E}_{G\sim\mathbb{D}}\ \mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)}\left[\underbrace{A_\lambda(\tau|\theta,G)}_{\text{advantage function}}\cdot\sum_{t=1}^{T}\underbrace{\nabla_\theta\log\Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G))}_{\text{score function}}\right], \tag{14}$$

Similar to Sec. II-C1, the gradient of the actor loss function in (9) is given by

$$\nabla_\theta J_\lambda(\theta) = \mathbb{E}_{G\sim\mathbb{D}}\ \mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)}\left[\sum_{t=1}^{T}\underbrace{A_{\lambda,t-1}(\tau|\theta,G)}_{\text{advantage function}}\cdot \underbrace{\nabla_\theta\log\Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G))}_{\text{score function}}\right], \tag{18}$$

where for $t=0,\ldots,T-1$,

$$A_{\lambda,t}(\tau|\theta,G) = R(\tau^t,G,\lambda) - V(\boldsymbol{x}^t,G). \tag{19}$$

We approximate the value function in (16) using a critic $V_\phi(\boldsymbol{x}^t,G)$ with learnable parameters ϕ. Let $J_V(\phi;\theta)$ denote the critic loss, where the actor parameters θ are held fixed when updating the critic. The gradient of the critic loss is then given by

$$\nabla_{\boldsymbol{\phi}} J_V(\boldsymbol{\phi},\boldsymbol{\theta}) = \mathbb{E}_{G\sim\mathbb{D}}\ \mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)} \left[\sum_{t=0}^{T-1}\Big(V_{\boldsymbol{\phi}}(\boldsymbol{x}^t,G) - R(\tau^t,G,\lambda)\Big)\cdot\nabla_{\boldsymbol{\phi}}V_{\boldsymbol{\phi}}(\boldsymbol{x}^t,G)\right]. \tag{20}$$

Detailed proof of (18)-(20) is given in Appendix A.

The overall L2A procedure for population-based GCO problems is summarized in Algorithm 1. During training, the outer loop gradually decreases the temperature parameter λ from 1 to 0, thereby reducing the contribution of the entropy regularizer. At each epoch, N_G graph samples approximate the outer expectation $\mathbb{E}_{G\sim\mathbb{D}}[\cdot]$ in (9). For each graph sample, the current policy Π_θ generates N_τ trajectories to approximate the inner expectation $\mathbb{E}_{\tau\sim\Pi_\theta(\cdot|\boldsymbol{Z}(G))}[\cdot]$. During inference, given a graph instance G and an initial solution $\boldsymbol{x}^0$, we recursively sample $\boldsymbol{x}^t\sim\Pi_\theta(\cdot\mid\boldsymbol{x}^{t-1},\boldsymbol{Z}(G))$ for $t=1,\cdots,T$ and output $\boldsymbol{x}^T$ as an approximation to $\boldsymbol{x}^*(G)$.

Algorithm 1 Generic Learning-to-Anneal Procedure

Input: graph distribution $\mathbb{D}$, objective function $f(\cdot,G)$, transition horizon T, graph number per batch N_G, trajectory number per graph N_τ, learning rate η
Training stage:
  Initialize the network parameter θ
  Fix the grouping of blockwise sampling as $\{1,\ldots,N\}=\bigcup_{m=1}^{M}\mathcal{I}_m$
  for $\ell=1,\ldots,L$ **do**
    Set $\lambda\leftarrow 1-\frac{\ell}{L}$
    Obtain N_G graph samples from distribution $G\sim\mathbb{D}$
    For each graph sample G, obtain N_τ trajectory samples $\{\tau_1,\ldots,\tau_{N_\tau}\}$
    For each trajectory $\tau=(\boldsymbol{x}^0,\cdots,\boldsymbol{x}^T)$, adopt the blockwise sampling at each step t with fixed grouping $\{\mathcal{I}_1,\ldots,\mathcal{I}_M\}$
    Compute $J_\lambda(\theta)$ in (9)
    $\theta\leftarrow\theta-\eta\ \nabla_\theta J_\lambda(\theta)$
  end for
  Output the trained network θ^*
Inference stage:
  Input: graph instance G
  Obtain a trajectory $\tau=(\boldsymbol{x}^0,\boldsymbol{x}^1,\cdots,\boldsymbol{x}^T)$ with $\boldsymbol{x}^t\sim\Pi_{\theta^*}(\cdot|\boldsymbol{x}^{t-1},Z(G))$, $t=1,\cdots,T$
  Output $\boldsymbol{x}^T$

III. Extended Transformer Policy Network

This section instantiates the stochastic transition policy in Sec. II-B by using neural networks. Following the organization of the generic L2A framework, we first motivate an evolving graph representation and then introduce the Transformer architecture that jointly samples the next solution and updates the graph embedding. We subsequently develop a blockwise autoregressive factorization that improves the expressivity of the solution sampler.

A. Updated Graph Embeddings Across Transitions

The basic L2A framework in Sec. II-B uses a fixed graph embedding $\boldsymbol{Z}(G)$ throughout a sampled trajectory. In practical GCO learning schemes, however, graph embeddings can be updated as the solution evolves. For example, the S2V-DQN model [22] recomputes graph embeddings according to the current partial solution so that the learned node representations reflect the evolving decision context. Moreover, [35] developed the stochastic computation graph framework, which jointly models deterministic feature transformations and stochastic sampling operations and provides unbiased gradient estimators for expected losses in such models. Motivated by those theoretical supports, we allow the graph embedding $\boldsymbol{Z}^t(G)$ to evolve after every transition step t. Therefore, the transition process in (41) can be rewritten as follows:

$$\begin{aligned}&[\boldsymbol{p}^t,\boldsymbol{Z}^t(G)] = \Pi_{\theta_t}\big(\cdot\mid\boldsymbol{x}^{t-1},\boldsymbol{Z}^{t-1}(G)\big),\\ &x_i^t\sim\text{Bernoulli}(p_i^t),\quad i\in V,\quad t=1,\ldots,T,\end{aligned} \tag{21}$$

where $\boldsymbol{p}^t \in (0,1)^N$ parameterizes the distribution of the next full-size solution. Note that the learnable parameters θ_t in (21) are also updated across transition steps, since the varying graph embeddings naturally lead to the evolving policies.

This extension does not invalidate the gradient-based training scheme in Sec. II-C. The evolving graph embedding can be viewed as a deterministic internal state of the policy network. Existing results on stochastic policies [45], [35] show that likelihood-ratio training schemes, such as the policy-gradient and actor-critic methods, remain valid and unbiased when deterministic, parameter-dependent internal states are embedded in the policy computation. Hence, updated graph embeddings constitute a valid practical extension of the training schemes in Sec. II-C.

B. Blockwise Autoregressive Sampling

In principle, the mean-field sampling for each solution (i.e., every bit is sampled in parallel) is fast but ignores conditional correlations among N bits. At the other extreme, bit-by-bit autoregressive sampling captures every conditional dependence but requires N sequential forward processes. We therefore introduce blockwise autoregressive sampling [37] as an efficient tradeoff between these two extreme cases. For simplicity, assume that N is divisible by M and we uniformly partition the full node set into M ordered blocks, i.e.,

$$\begin{aligned}&\{1,\dots,N\} = \bigcup_{m=1}^{M} \mathcal{I}_m,\\ &|\mathcal{I}_m| = B \triangleq N/M,\ \forall m = 1,\dots,M,\\ &\mathcal{I}_m \cap \mathcal{I}_{m'} = \emptyset,\ m \neq m'.\end{aligned} \tag{22}$$

Define $\mathcal{I}_{<m} = \bigcup_{j=1}^{m-1} \mathcal{I}_j$. Before sampling block $\mathcal{I}_m$ at transition t, we construct the partial current solution as follows:

$$\widetilde{x}_i^{t,m-1} = \begin{cases} x_i^t, & i \in \mathcal{I}_{<m},\\ \mathrm{UAS}, & i \in \{1,\dots,N\} \setminus \mathcal{I}_{<m}. \end{cases} \tag{23}$$

where UAS represents the **unassigned state** for every bit throughout this paper. Conditioned on $\widetilde{\boldsymbol{x}}^{t,m-1}$, the complete previous solution $\boldsymbol{x}^{t-1}$, and the current graph embedding $\boldsymbol{Z}^{t,m-1}(G)$, the policy samples all variables in the next block in parallel:

$$\begin{aligned}&[\boldsymbol{p}^{t,m}, \boldsymbol{Z}^{t,m}(G)] = \Pi_{\theta_t}\big(\cdot \mid \widetilde{\boldsymbol{x}}^{t,m-1}, \boldsymbol{x}^{t-1}, \boldsymbol{Z}^{t,m-1}(G)\big),\\ &x_i^t \sim \mathrm{Bernoulli}(p_i^{t,m}),\ i \in \mathcal{I}_m,\end{aligned} \tag{24}$$

with $\boldsymbol{Z}^{t,0}(G) = \boldsymbol{Z}^{t-1}(G)$. After M block evaluations, $\widetilde{\boldsymbol{x}}^{t,M} = \boldsymbol{x}^t$ and $\boldsymbol{Z}^{t,M}(G) = \boldsymbol{Z}^t(G)$. Equivalently, the transition probability factorizes as

$$\begin{aligned}&\Pi_{\theta_t}(\boldsymbol{x}^t \mid \boldsymbol{x}^{t-1}, \boldsymbol{Z}^{t-1}(G))\\ &\quad= \prod_{m=1}^{M} \Pi_{\theta_t}\big(\boldsymbol{x}^t_{\mathcal{I}_m} \mid \widetilde{\boldsymbol{x}}^{t,m-1}, \boldsymbol{x}^{t-1}, \boldsymbol{Z}^{t,m-1}(G)\big),\end{aligned} \tag{25}$$

where $\boldsymbol{x}^t_{\mathcal{I}_m} \triangleq \{x_i^t \mid i \in \mathcal{I}_m\}$ and $\boldsymbol{x}^t \triangleq \cup_{m=1}^{M} \boldsymbol{x}^t_{\mathcal{I}_m}$. The special case $M = 1$ reduces to a mean-field sampler, whereas $M = N$ reduces to a bit-by-bit autoregressive sampler. Intermediate values of M preserve parallel sampling within each uniformly sized block while modeling dependencies across blocks. In this case, the same policy network in Sec. III-C is reused for the M conditional samplings, with the partial current solution appended as an additional conditioning sequence.

C. Transformer Architecture

We now design an extended Transformer to instantiate the blockwise sampling-based transition policy in (24). Importantly, the blockwise autoregressive sampling method and the updated graph embedding are adopted. The three node assignment states, i.e., "0", "1", and "UAS", are represented by the following vectors

$$\boldsymbol{e}_0 = \begin{bmatrix}1\\0\end{bmatrix}, \quad \boldsymbol{e}_1 = \begin{bmatrix}0\\1\end{bmatrix}, \quad \boldsymbol{e}_{\mathrm{UAS}} = \begin{bmatrix}0\\0\end{bmatrix}. \tag{26}$$

Let $\boldsymbol{X}^{t-1} \in \mathbb{R}^{2\times N}$ denote the assignment embedding of the previous solution $\boldsymbol{x}^{t-1}$, i.e., $\boldsymbol{X}^{t-1}(:,i) = \boldsymbol{e}_{x_i^{t-1}}$ for $i = 1,\dots,N$. Let $\tilde{\boldsymbol{X}}^{t,m-1} \in \mathbb{R}^{2\times N}$ denote the assignment embedding of the current solution before sampling the block $\mathcal{I}_m$, i.e.,

$$\tilde{\boldsymbol{X}}^{t,m-1}(:,i) = \begin{cases} \boldsymbol{e}_{x_i^t}, & i \in \mathcal{I}_{<m},\\ \boldsymbol{e}_{\mathrm{UAS}}, & i \in \{1,\dots,N\} \setminus \mathcal{I}_{<m}. \end{cases} \tag{27}$$

In addition, let $\boldsymbol{Z}^{t,m-1}(G) \in \mathbb{R}^{D\times N}$ denote the updated graph embedding before sampling the block $\mathcal{I}_m$, and let $\boldsymbol{Z}^{(e)} \in \mathbb{R}^{D\times N}$ denote the initial graph embedding. We construct the following two input token sequences:

$$\begin{aligned}\boldsymbol{X}_q^{t,m} &= \left[\tilde{\boldsymbol{X}}^{t,m-1}; \boldsymbol{X}^{t-1}; \bar{\boldsymbol{Z}}^{t,m-1}(G)\right] \in \mathbb{R}^{(D+4)\times N},\\ \boldsymbol{X}_k^{t,m} &= \left[\tilde{\boldsymbol{X}}^{t,m-1}; \boldsymbol{X}^{t-1}; \boldsymbol{Z}^{(e)}\right] \in \mathbb{R}^{(D+4)\times N},\end{aligned} \tag{28}$$

where $\bar{\boldsymbol{Z}}^{t,m-1}(G) \triangleq \mathrm{LayerNorm}(\boldsymbol{Z}^{t,m-1}(G))$. Thus, each node-wise token vector contains the discrete assignment embeddings and the continuous graph embedding.

Based on (28), the extended Transformer applies multi-head attention using $\boldsymbol{X}_q^{t,m}$ as the query token sequence and $\boldsymbol{X}_k^{t,m}$ as the key-value token sequence. For each attention head $h = 1,\dots,H$,

$$\begin{aligned}\boldsymbol{Q}^{(h)} &= (\boldsymbol{X}_q^{t,m})^\top \boldsymbol{W}_q^{(h)} \in \mathbb{R}^{N\times D_q/H},\\ \boldsymbol{K}^{(h)} &= (\boldsymbol{X}_k^{t,m})^\top \boldsymbol{W}_k^{(h)} \in \mathbb{R}^{N\times D_k/H},\\ \boldsymbol{V}^{(h)} &= (\boldsymbol{X}_k^{t,m})^\top \boldsymbol{W}_v^{(h)} \in \mathbb{R}^{N\times D_v/H},\end{aligned} \tag{29}$$

where $\boldsymbol{W}_q^{(h)} \in \mathbb{R}^{(D+4)\times(D_q/H)}$, $\boldsymbol{W}_k^{(h)} \in \mathbb{R}^{(D+4)\times(D_k/H)}$, and $\boldsymbol{W}_v^{(h)} \in \mathbb{R}^{(D+4)\times(D_v/H)}$. Typically, $D_q = D_k$. The output of the hth attention head is

$$\boldsymbol{A}^{(h)} = \mathrm{softmax}\left(\frac{\boldsymbol{Q}^{(h)}\boldsymbol{K}^{(h)\top}}{\sqrt{D_k/H}}\right)\boldsymbol{V}^{(h)} \in \mathbb{R}^{N\times(D_v/H)}. \tag{30}$$

The outputs of the H attention heads are concatenated and projected as

$$\boldsymbol{Y}^{t,m} = \mathrm{MHA}\left(\boldsymbol{X}_q^{t,m}, \boldsymbol{X}_k^{t,m}\right) = \boldsymbol{W}_o[\boldsymbol{A}^{(1)},\dots,\boldsymbol{A}^{(H)}]^\top, \tag{31}$$

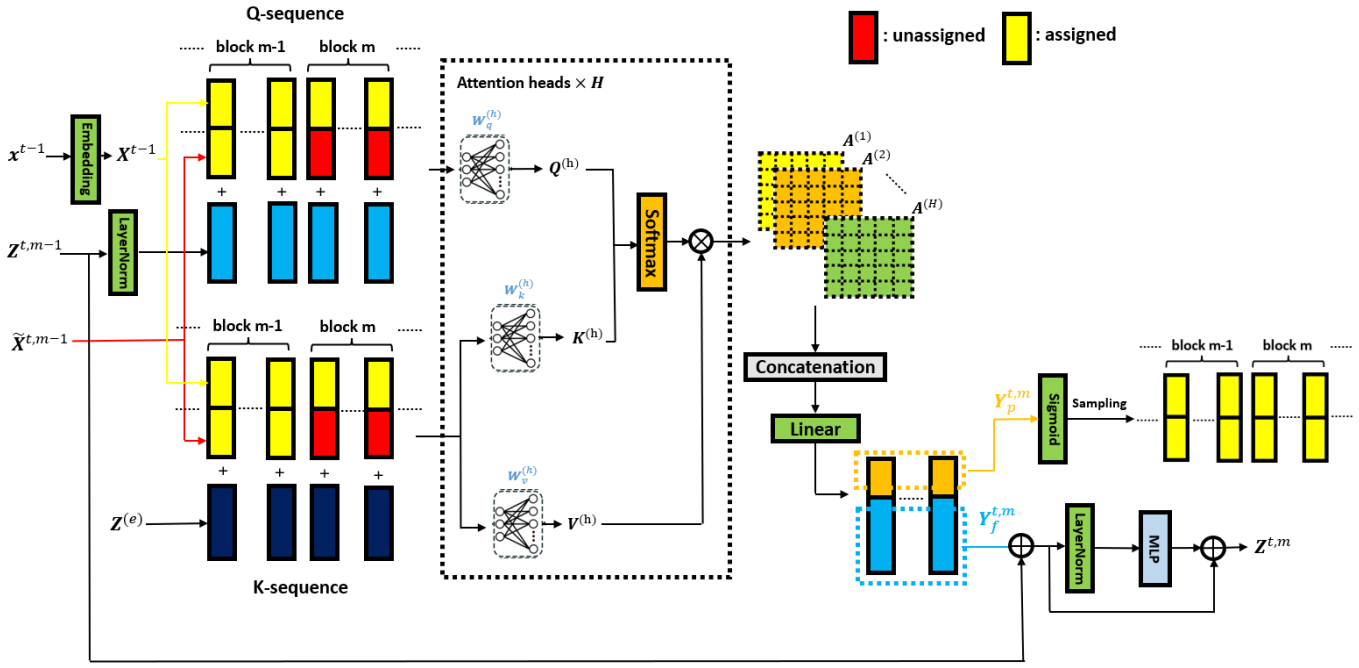


Fig. 2. The architecture of each Transformer policy network with blockwise auto-regressive sampling method.

where $\boldsymbol{W}_o \in \mathbb{R}^{(D+1)\times D_v}$ and $\boldsymbol{Y}^{t,m} \in \mathbb{R}^{(D+1)\times N}$. We divide the output into a solution-logit component and a graph-feature component:

$$\boldsymbol{Y}^{t,m} = [\boldsymbol{Y}_p^{t,m}; \boldsymbol{Y}_f^{t,m}],\ \boldsymbol{Y}_p^{t,m} \in \mathbb{R}^{1\times N},\ \boldsymbol{Y}_f^{t,m} \in \mathbb{R}^{D\times N}. \tag{32}$$

The sampling probability vector is then given by

$$\boldsymbol{p}^{t,m} = \sigma(\boldsymbol{Y}_p^{t,m}) \in (0,1)^N. \tag{33}$$

The graph embedding is updated (i.e., $\boldsymbol{Z}^{t,m-1}(G) \to \boldsymbol{Z}^{t,m}(G)$) through residual connections and an MLP:

$$\begin{aligned}\tilde{\boldsymbol{Z}}^{t,m}(G) &= \boldsymbol{Z}^{t,m-1}(G) + \boldsymbol{Y}_f^{t,m},\\ \boldsymbol{Z}^{t,m}(G) &= \tilde{\boldsymbol{Z}}^{t,m}(G) + \mathrm{MLP}(\mathrm{LayerNorm}(\tilde{\boldsymbol{Z}}^{t,m}(G))).\end{aligned} \tag{34}$$

Although the policy network outputs N probabilities in every blockwise sampling step, **only the entries in the current block are sampled**, i.e., $x_i^t \sim \mathrm{Bernoulli}(p_i^{t,m})$ for $i \in \mathcal{I}_m$. Therefore we obtain:

$$\tilde{\boldsymbol{X}}^{t,m}(:,i) = \begin{cases} \boldsymbol{e}_{x_i^t}, & i \in \mathcal{I}_{<m+1},\\ \boldsymbol{e}_{\mathrm{UAS}}, & i \in \{1,\dots,N\} \setminus \mathcal{I}_{<m+1}. \end{cases} \tag{35}$$

The updated solution $\tilde{\boldsymbol{X}}^{t,m}$ and the updated graph embedding $\boldsymbol{Z}^{t,m}(G)$ are then passed to the next blockwise autoregressive sampling as conditional variables. Consequently, we obtain the final results for the transition step t:

$$\boldsymbol{X}^t \triangleq \tilde{\boldsymbol{X}}^{t,M} \in \mathbb{R}^{2\times N},\ \boldsymbol{Z}^t(G) \triangleq \boldsymbol{Z}^{t,M}(G) \in \mathbb{R}^{D\times N}. \tag{36}$$

IV. Random Masking as a Curriculum Learning Method

Random masking [11], [18] has been widely used in model training to hide part of the inputs, forcing the model to learn more robust feature embeddings from incomplete observations. In our work, random masking controls the number of activated graph nodes. Let

$$\mathcal{N} \triangleq \{N_1, N_2, \dots, N_L\},\ 0 < N_1 < \cdots < N_L = N, \tag{37}$$

denote a node-number schedule, which means that $N - N_\ell$ graph nodes are randomly selected and masked at the ℓth training stage. For each graph sample $G = (V, E, w)$, we obtain the following nested unmasked node sets

$$\mathcal{U}_1(G) \subset \cdots \subset \mathcal{U}_L(G) = V = \{1,\dots,N\}, \tag{38}$$

where $|\mathcal{U}_\ell(G)| = N_\ell$ for $\ell = 1,\dots,L$. The complementary mask set is

$$\mathcal{M}_\ell(G) \triangleq V \setminus \mathcal{U}_\ell(G). \tag{39}$$

At the ℓth training stage, the same node mask $(\mathcal{U}_\ell(G), \mathcal{M}_\ell(G))$ is adopted by all trajectories sampled from graph G. Therefore, for all transition steps $t = 1,\dots,T$, the solution $\boldsymbol{x}^t = \{x_1^t,\dots,x_N^t\}$ satisfies

$$x_i^t = \mathrm{MASK},\ \forall i \in \mathcal{M}_\ell(G). \tag{40}$$

Hence the transition policy directly samples the next unmasked assignments based on the previous unmasked assignments, given as follows:

$$\begin{aligned}\boldsymbol{p}_{\mathcal{U}_\ell}^t &= \Pi_{\theta_t}\left(\cdot \mid \boldsymbol{x}_{\mathcal{U}_\ell}^{t-1}, \boldsymbol{Z}^t(G)\right),\\ \boldsymbol{x}_{\mathcal{U}_\ell}^t &\sim \boldsymbol{p}_{\mathcal{U}_\ell}^t,\ t = 1,\dots,T,\end{aligned} \tag{41}$$

where $\boldsymbol{x}^t_{\mathcal{U}_\ell} \triangleq \{x^t_i \mid i \in \mathcal{U}_\ell\}$. As ℓ increases, more graph nodes become unmasked and participate in the optimization, which means that the policy network needs to learn a more complex topology. Hence, the schedule in (37) implements an **easy-to-hard curriculum learning** over the problem dimensions. At the final stage, $\mathcal{U}_L(G) = V$, and the training problem coincides with the original N-node GCO problem.

The Transformer policy network with random masking at each transition step t is illustrated in Fig. 2. It shows that if the input (node) masking vector is $\boldsymbol{n}$, where each entry is either 0 (activated) or $-\infty$ (masked), the effective attention masking matrix is given by $\mathcal{M} \triangleq \boldsymbol{n}\boldsymbol{n}^\top \in \mathbb{R}^{N\times N}$, or equivalently, defined as follows:

$$\mathcal{M}(i,j) = \begin{cases} -\infty, & i \in \boldsymbol{n} \text{ or } j \in \boldsymbol{n}, \\ 0, & \text{otherwise.} \end{cases} \tag{42}$$

V. Proposed Curriculum Learning

A. Filter Function-Enhanced Curriculum Learning

Vanilla curriculum learning (CL) [7] improves training by presenting the model with a sequence of increasingly difficult optimization criteria. It typically begins with an easier surrogate objective and gradually transitions to the original objective of interest. A representative example is to first optimize a smoothed version of a non-convex objective and gradually consider less smoothing, with the intuition that a smooth version of the problem reveals the global picture. Consider a function $f_\lambda(\cdot)$ with λ decreasing from 1 to 0, where $f_1(\cdot)$ is highly smoothed and easy to optimize and $f_0(\cdot)$ is the original problem. One first minimizes $f_1(\cdot)$ and then gradually decreases λ. Such a vanilla CL can be realized by adding an entropy regularization term to the loss function, given as follows

$$J_\lambda(\theta;\tau,G) = g(\tau,G) + \lambda \ \log p(\tau \mid \theta, \boldsymbol{Z}(G)), \tag{43}$$

where the loss function gradually transitions from $J_1(\theta;\tau,G)$ to $J_0(\theta;\tau,G)$. The temperature parameter λ in (43) balances exploration and exploitation. However, when the objective $g(\cdot)$ is highly spiky, the entropy term cannot address the issue of poor convergence [2] since it is only an additive term in (43).

To address this issue, inspired by [10], [15], we adopt the **filter function as a stronger smoothing operator**. This leads to an enhanced CL approach to highly non-convex optimization problems. In fact, filter functions have been widely used in signal processing, e.g., for denoising in image processing [16] and trajectory smoothing in time-series estimation [14], where they suppress high-frequency fluctuations and yield smoother effective signals. Analogously, the original loss landscape in our formulation is highly spiky, with many sharp local optima that can trap the learned policy. By applying the filter function, the effective landscape is smoothed, transforming narrow local pitfalls into wider and less abrupt basins. This facilitates escaping poor local regions, leading to more stable training and improved solution quality.

In particular, let $\boldsymbol{x}^t$ denote the sampled solution at the tth optimization step, and let $\mathcal{N}(\boldsymbol{x}^t)$ denote the neighborhood of $\boldsymbol{x}^t$. A filter function can be defined as follows

$$S(\boldsymbol{x}^t) = \arg\min_{\boldsymbol{x}\in\mathcal{N}(\boldsymbol{x}^t)} f(\boldsymbol{x}, G), \tag{44}$$

where $f(\boldsymbol{x}, G)$ is the original Max-Cut objective in (2). This shows that the filter function $S(\cdot)$ acts as an amortized post-processing operator — it maps all the nearby sampled solutions to the best solution in the neighborhood. This operation attenuates the abrupt local variations and consequently, sharp local optima are smoothed into wider basins, leading to a more regular and well-behaved effective landscape. For notational simplicity, define a filtered solution trajectory as follows:

$$S(\tau) \triangleq (S(\boldsymbol{x}^0), S(\boldsymbol{x}^1), \ldots, S(\boldsymbol{x}^T)). \tag{45}$$

In principle, there are two ways to incorporate the filter function into the loss in (43). The first is to let the filter act only on the original objective in (2), i.e.,

$$J_{\lambda,S}(\theta;\tau,G) = (f \circ S)(\tau,G) + \lambda \ \log p(\tau \mid \theta, \boldsymbol{Z}(G)), \tag{46}$$

hence the original objective $f(\cdot)$ is replaced by the **composite objective** $(f \circ S)(\cdot)$, while the policy network at layer t still takes the unfiltered solution $\boldsymbol{x}^{t-1}$ as the input. The second is to let the filter act on both the original objective and the sampled trajectory

$$J_{\lambda,S}(\theta;\tau,G) = f(S(\tau),G) + \lambda \ \log p(S(\tau) \mid \theta, \boldsymbol{Z}(G)). \tag{47}$$

This can be viewed as another enhanced variant of (43), in which the original **sampled trajectory** τ is replaced by its filtered counterpart $S(\tau)$. In contrast to (46), here the policy network at layer t takes the filtered solution $S(\boldsymbol{x}^{t-1})$ as the input. It should be noted that

$$(f \circ S)(\tau,G) = f(S(\tau),G) = \sum_{t=0}^{T} f(S(\boldsymbol{x}^t),G). \tag{48}$$

B. Policy Gradient for Proposed Curriculum Learning

Let the parameter m control the strength of the filter function, such that a larger m corresponds to stronger filtering in (44). For clarity, the filter function is denoted by $S_m(\cdot)$ in the sequel. Consequently, we introduce a **new curriculum** in which both the parameters λ and m are gradually reduced accordingly. Based on the two loss functions in (46) and (47) that incorporate the filtering operations, we obtain the corresponding two policy-gradient variants of the vanilla L2A framework described in Fig. 1.

1) Learning with Enhanced Objective: If the filter function is adopted in an L2A framework according to (46), the original objective is enhanced from $f(\cdot)$ to $f \circ S_m(\cdot)$. The corresponding enhanced L2A framework and its geometric interpretation are illustrated in Fig. 3 and Fig. 5, respectively. Therefore, the expected loss over the graph distribution can be expressed as (49). Similar to (14), the gradient of (49) is computed by

$$\nabla_\theta J_{\lambda,S_m}(\theta) = \mathbb{E}_{G\sim\mathbb{D}} \ \mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)} \Bigg[\underbrace{A_{\lambda,m}(\tau|\theta,G)}_{\text{advantage function}} \cdot \sum_{t=1}^{T} \underbrace{\nabla_\theta \log \Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1}, \boldsymbol{Z}(G))}_{\text{score function}} \Bigg], \tag{50}$$

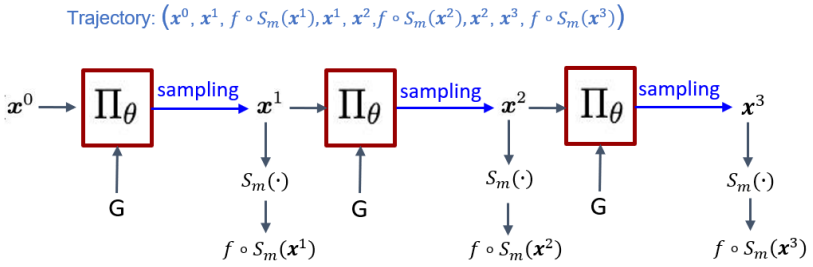


Fig. 3. Enhanced L2A framework trained on the original solution trajectory.

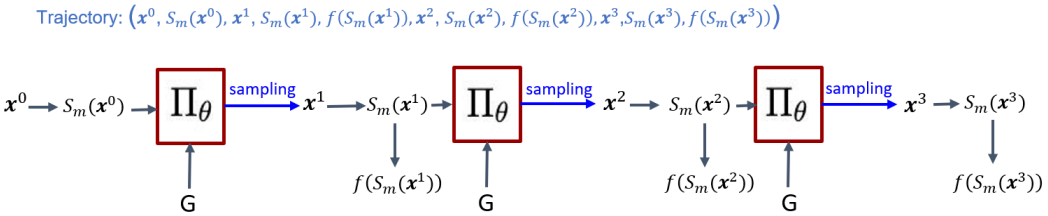


Fig. 4. Enhanced L2A framework trained on the solution trajectory after filtering.

where

$$A_{\lambda,m}(\tau|\theta,G) = (f \circ S_m)(\tau,G) + \lambda \log p(\tau|\theta,\boldsymbol{Z}(G)) - b(G),$$
$$b(G) = \mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)}\left[(f \circ S_m)(\tau,G) + \lambda \log p(\tau|\theta,\boldsymbol{Z}(G))\right]. \tag{51}$$

As seen in (50), the filter function modifies the advantage term by using the filtered objective values, while the score term remains the same as in the vanilla policy-gradient result (14).

2) Learning with Post-processed Trajectories: If the filter function is applied according to (47), the trajectory τ is replaced by the post-processed trajectory $S_m(\tau)$. The corresponding enhanced L2A framework and its geometric interpretation are illustrated in Fig. 4 and Fig. 6, respectively. Therefore, the expected loss over the graph distribution can be expressed as (52). Similar to (14), the gradient of (52) is computed by

$$\nabla_\theta J_{\lambda,S_m}(\theta) = \mathbb{E}_{G\sim\mathbb{D}}\ \mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)}\Bigg[\underbrace{A_{\lambda,m}(S_m(\tau)|\theta,G)}_{\text{advantage function}} \cdot \sum_{t=1}^{T} \underbrace{\nabla_\theta \log \Pi_\theta(\boldsymbol{x}^t|S_m(\boldsymbol{x}^{t-1}),\boldsymbol{Z}(G))}_{\text{score function}}\Bigg], \tag{53}$$

where

$$A_{\lambda,m}(S_m(\tau)|\theta,G) = f(S_m(\tau),G) + \lambda \log p(S_m(\tau)|\theta,\boldsymbol{Z}(G)) - b(G),$$
$$b(G) = \mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)}\left[f(S_m(\tau),G) + \lambda \log p(S_m(\tau)|\theta,\boldsymbol{Z}(G))\right]. \tag{54}$$

As seen in (53), the filter function modifies both the advantage and the score terms.

C. Two Filter Function Examples

In this subsection, we present two local-search methods as practical examples of filter functions. Local search is a standard tool in discrete optimization: given a current solution, it generally returns an improved solution in its neighborhood region.

1) Greedy Local Search: We first introduce a greedy local-search method as the filter function, which is inspired by [10]. The parameter m, which controls the search strength of the filter function, is mainly reflected in the perturbation rounds of the greedy local search. A larger m leads to a stronger local-improvement effect and therefore a smoother effective training target.

A T-layer policy network is employed. At the tth layer, let $\{\boldsymbol{x}_k^t\}_{k=1}^K$ denote the K sampled solutions maintained by the parallel simulators. For each layer t and branch k, the local-search module outputs a refined solution $S_m(\boldsymbol{x}_k^t)$. Specifically, initialize $\boldsymbol{z}_k^{(t,0)} = \boldsymbol{x}_k^t$, then we perform m rounds of stochastic perturbation over N nodes. In every round, we compute a node-wise flip score for each node. The nodes are then ranked according to the resulting flip scores, and a subset consisting of the top-K_{f} nodes is selected, where K_{f} denotes the number of nodes flipped at each perturbation step. The selected nodes are flipped simultaneously to form a candidate solution, which is accepted only if it improves the objective value (in a greedy manner). Specifically, for the kth simulator at the tth layer and the ℓth perturbation step, denote the index set of the flipped nodes as $\Omega_k^{(t,\ell)}$, where $\Omega_k^{(t,\ell)} \subset \{1,\ldots,N\}$. Then we obtain (55), which is the candidate solution after the perturbation. The candidate is accepted only if it improves the objective value,

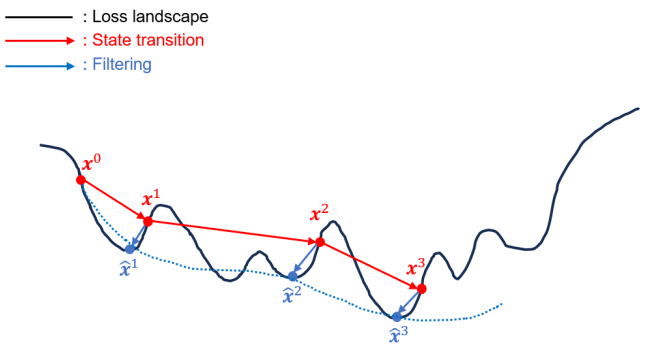


Fig. 5. Geometry of the enhanced L2A on the original solution trajectory.

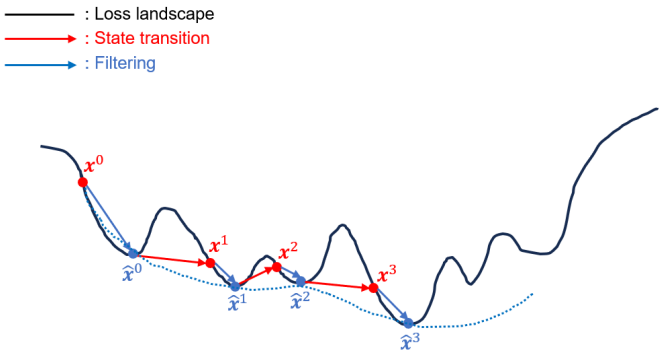


Fig. 6. Geometry of the enhanced L2A on the solution trajectory after filtering.

$$J_{\lambda,S_m}(\theta) \triangleq \mathbb{E}_{G\sim\mathbb{D}}\ \mathbb{E}_{\tau=(\boldsymbol{x}^0,\boldsymbol{x}^1,\cdots,\boldsymbol{x}^T):\boldsymbol{x}^t\sim\Pi_\theta(\cdot|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G))}\left[J_{\lambda,S_m}(\theta;\tau,G)\right],$$
$$= \mathbb{E}_{G\sim\mathbb{D}}\left[\int_\tau \left(f\circ S_m(\tau,G)+\lambda\log p(\tau|\theta,\boldsymbol{Z}(G))\right)p(\tau|\theta,\boldsymbol{Z}(G))d\tau\right]. \tag{49}$$

$$J_{\lambda,S_m}(\theta) \triangleq \mathbb{E}_{G\sim\mathbb{D}}\ \mathbb{E}_{S_m(\tau)=(S_m(\boldsymbol{x}_0),\ldots,S_m(\boldsymbol{x}^T)):\boldsymbol{x}^t\to S_m(\boldsymbol{x}^t),\ \boldsymbol{x}^t\sim\Pi_\theta(\cdot|S_m(\boldsymbol{x}^{t-1}),\boldsymbol{Z}(G))}\left[J_{\lambda,S_m}(\theta;\tau,G)\right],$$
$$= \mathbb{E}_{G\sim\mathbb{D}}\left[\int_\tau \left(f(S_m(\tau),G)+\lambda\ \log p(S_m(\tau)|\theta,\boldsymbol{Z}(G))\right)\ p(S_m(\tau)|\theta,\boldsymbol{Z}(G))d\tau\right]. \tag{52}$$

i.e.,

$$\boldsymbol{z}_k^{(t,\ell+1)} = \begin{cases} \tilde{\boldsymbol{z}}_k^{(t,\ell)}, & f(\tilde{\boldsymbol{z}}_k^{(t,\ell)},G)\le f(\boldsymbol{z}_k^{(t,\ell)},G), \\ \boldsymbol{z}_k^{(t,\ell)}, & \text{otherwise.}\end{cases} \tag{56}$$

Hence, a local search is performed via multi-node perturbations in (55) and a greedy acceptance rule in (56), where $\ell\in\{1,\ldots,m\}$. It finally outputs the solutions $S_m(\boldsymbol{x}_k^t)=\boldsymbol{z}_k^{(t,m)}$, for $k=1,\ldots,K$.

Notably, if $\boldsymbol{x}^\star(G)$ is a globally optimal solution, then the above filter preserves optimality in objective value, provided that only non-deteriorating updates are accepted. In this case,

$$f(\boldsymbol{x}^\star(G),G)=f(S_m(\boldsymbol{x}^\star(G)),G). \tag{57}$$

Therefore, the filter improves non-optimal samples while leaving optimal solutions unchanged in value.

2) Local Search Based on Slowest Descent: We next introduce a more structured local-search strategy based on K simulators. Analogous to the slowest descent scheme in [36], we relax the feasible region in the original GCO problem (1) from $\boldsymbol{x}\in\{0,1\}^N$ to $\boldsymbol{x}\in[0,1]^N$. Under this relaxation, the analytic continuation of the objective function, denoted by $\tilde{f}(\boldsymbol{x},G)$, becomes differentiable and smooth over the continuous domain $\boldsymbol{x}\in[0,1]^N$. For example, for the Max-Cut objective $f(\boldsymbol{x},G)=-\sum_{i<j}\frac{1}{2}(1-x_ix_j)w(i,j)$, the same analytic form is differentiable and smooth on $[0,1]^N$.

To avoid numerical ill-conditioning near the boundary of the hypercube, for the kth simulator, we add a small perturbation on the policy network's output $\boldsymbol{x}_k^t\in\{0,1\}^N$ and obtain the

$$\left\{\tilde{\boldsymbol{z}}_k^{(t,\ell)} \in \{0,1\}^N : \tilde{z}_k^{(t,\ell)}(n) = \begin{cases} 1 - z_k^{(t,\ell)}(n), & n \in \Omega_k^{(t,\ell)} \\ z_k^{(t,\ell)}(n), & n \notin \Omega_k^{(t,\ell)} \end{cases}\right\}_{n=1}^{N}, \tag{55}$$

following solution:

$$\left\{\tilde{\boldsymbol{x}}_k^t \in \mathbb{R}^N : \tilde{x}_k^t(n) = \begin{cases} \epsilon, & x_k^t(n) = 0 \\ 1-\epsilon, & x_k^t(n) = 1 \end{cases}\right\}_{n=1}^{N}, \tag{58}$$

for all $n \in [N]$ and $\epsilon \ll 1$. We start from $\tilde{\boldsymbol{x}}_k^t$ and perform gradient descent on the continuous objective $\tilde{f}(\boldsymbol{x}, G)$ to obtain the stationary point $\dot{\boldsymbol{x}}_k^t$, and then compute the Hessian matrix at $\dot{\boldsymbol{x}}_k^t$ as follows

$$\boldsymbol{H}_x \triangleq \nabla_{\boldsymbol{x}}^2 \tilde{f}(\dot{\boldsymbol{x}}_k^t, G). \tag{59}$$

Let $\{\beta_i\}_{i=1}^N$ and $\{\boldsymbol{v}_i\}_{i=1}^N$ denote the eigenvalues and corresponding eigenvectors of $\boldsymbol{H}_x \in \mathbb{R}^{N\times N}$, respectively. According to the slowest descent method in [36], to reduce the computational complexity, we select Q eigenvectors of the Hessian matrix $\boldsymbol{H}_x$ corresponding to the Q smallest eigenvalues, denoted by $(\boldsymbol{v}_1^\star, \boldsymbol{v}_2^\star, \ldots, \boldsymbol{v}_Q^\star)$ and $(\beta_1^\star, \beta_2^\star, \ldots, \beta_Q^\star)$. For each $\boldsymbol{v}_j^\star$, $j \in [Q]$, we construct a set of neighboring candidates along the ray

$$\boldsymbol{y}_j(\mu, \dot{\boldsymbol{x}}_k^t) = \dot{\boldsymbol{x}}_k^t + \mu \cdot \boldsymbol{v}_j^\star, \quad \mu \geq 0. \tag{60}$$

To map this continuous ray back to discrete candidates, we define $\mu^{(n)} \triangleq \frac{0.5 - \dot{\boldsymbol{x}}_k^t(n)}{\boldsymbol{v}_j^\star(n)}$ as the flip point of the nth node, for all $n \in [N]$ satisfying $\boldsymbol{v}_j^\star(n) \cdot (0.5 - \dot{\boldsymbol{x}}_k^t(n)) > 0$. The values $\{\mu^{(n)}\}_{n=1}^N$ indicate when the corresponding coordinate crosses the rounding threshold 0.5 and flips the node. The m neighboring solutions of $\dot{\boldsymbol{x}}_k^t$ along the ray $\boldsymbol{y}_j(\mu, \dot{\boldsymbol{x}}_k^t)$ are then generated through the following sequential node-flip procedure:

1). Obtain the closest discrete solution $\boldsymbol{x}_k^{t,j,0}$ via the symbol slicing of $\dot{\boldsymbol{x}}_k^t$.
2). Obtain the ascending order $0 < \mu^{(n_1)} < \cdots < \mu^{(n_{m-1})} < \ldots$.
3). Flip the n_1^{th} node of the solution $\boldsymbol{x}_k^{t,j,0}$ to obtain the second neighboring solution $\boldsymbol{x}_k^{t,j,1}$.
4). Iterate step 3) for $m-1$ times: at the ℓth iteration, flip the n_ℓ^{th} bit of $\boldsymbol{x}_k^{t,j,\ell-1}$ to obtain the neighboring solution $\boldsymbol{x}_k^{t,j,\ell}$.

Consequently, we obtain m solution candidates $\{\boldsymbol{x}_k^{t,j,0}, \ldots, \boldsymbol{x}_k^{t,j,m-1}\}$ along the ray $\boldsymbol{y}_j(\mu, \dot{\boldsymbol{x}}_k^t)$. Finally, we select the best solution among all mQ solution candidates along the Q rays $\{\boldsymbol{y}_1(\mu, \dot{\boldsymbol{x}}_k^t), \ldots, \boldsymbol{y}_Q(\mu, \dot{\boldsymbol{x}}_k^t)\}$, which is the output of the local-search function, i.e.,

$$S_m(\boldsymbol{x}_k^t) = \arg\min_{\boldsymbol{x}\in\mathcal{C}_k^t} f(\boldsymbol{x}, G), \tag{61}$$

where $\mathcal{C}_k^t \triangleq \{\boldsymbol{x}_k^{t,j,\ell} \mid j = 1, \ldots, Q,\ \ell = 0, \ldots, m-1\}$ is the candidate set. In general, this slowest descent-based scheme performs local search over far fewer neighboring solutions than the greedy local search in Sec. V-C1 by effectively exploiting meaningful geometric information from the relaxed objective.

VI. Performance Evaluation

A. Experiment Setup

Instance generation. We evaluate the proposed L2A scheme on the Max-Cut problem in (2) and compare it with several baseline methods. Our scheme is trained on three graph distributions summarized in Table I, including Erdős–Rényi (ER), Barabási–Albert (BA), and Power-Law (PL) graphs. For ER graphs, each edge is independently included with probability $p = 0.15$; For BA graphs, each newly added node is connected to $m = 4$ existing nodes following the preferential-attachment mechanism. For PL graphs, we use the default setting of the standard power-law graph generator.

TABLE I
GRAPH DISTRIBUTIONS AND THEIR PARAMETERS.

Distribution	Abbreviation	Key Parameter
Erdős–Rényi	ER	$p = 0.15$
Barabási–Albert	BA	$q = 4$
Power Law	PL	Default

Training, validation, and test. The proposed model is trained on graphs sampled from the distributions in Table I, with a training batch size of 16. The number of nodes varies from 100, 200 to 1200. To mitigate overfitting, for each graph size during training, we construct a validation set consisting of 100 held-out graphs sampled from the same training distribution. We then report the test results on graph set with number of nodes varying from 100, 200, to 1200, where each graph size contains 30 test instances.

L2A network. The main hyperparameters of the proposed scheme are summarized in Table II. The proposed L2A framework consists of two components: a graph-embedding GNN that pre-processes a graph G into its (normalized) embedding vectors, and a Transformer-based policy network.

Specifically, we pretrain the GNN using an unsupervised learning method named Large-scale Information Network Embedding (LINE) [40] [1]. The inputs are the number of nodes N and an edge list of graph G (using two directed edges to represent an undirected edge), while output is $\boldsymbol{Z}(G) \in \mathbb{R}^{D\times N}$, where each node i has an embedding vector $\boldsymbol{z}_i(G) \in \mathbb{R}^D$. We use a standard graph convolutional network (GCN) in PyTorch, which consists of one linear layer as the input layer, two convolutional layers in the middle, and another linear layer as the output layer as follows:

- For each node i, we initialize a random embedding vector $\boldsymbol{z}_i^0(G) \in \mathbb{R}^D$.
- The input linear layer yields $\boldsymbol{h}_i^0 = \boldsymbol{W}^0 \boldsymbol{z}_i^0(G)$, where $\boldsymbol{W}^0 \in \mathbb{R}^{D\times D}$ is the input projection matrix.

[1] LINE codes: https://github.com/tangjianpku/LINE

TABLE II
HYPERPARAMETERS OF THE PROPOSED L2A SCHEME.

Hyperparameter	Value
Graph samples per epoch N_G	8
Sampled trajectories per graph N_τ	32
Learning rate η	10^{-4}
Layer number of policy networks T	16
Embedding dimension D	256
Attention head number H	8
Greedy local-search step number m	1
Slowest-descent local-search step number m	1
Selected descent directions number Q	4

- Two convolutional layers are functioned as follows

$$\boldsymbol{h}_i^k = \sigma\left(\boldsymbol{W}^k \sum_{j\in\mathcal{N}(i)} \frac{\boldsymbol{h}_u^{k-1}}{|\mathcal{N}(i)|} + (\boldsymbol{b}^k)^T \boldsymbol{h}_v^{k-1}\right),\ k=1,2, \tag{62}$$

  where $\boldsymbol{b}^k \in \mathbb{R}^D$, $\sigma(\cdot)$ denotes an activation function, e.g., the ReLU activation function, and the set $\mathcal{N}(i)$ denotes all neighboring nodes of each node i.
- The output linear layer yields the initial graph embedding $\boldsymbol{x}_i^{(e)} = \boldsymbol{W}^3 \boldsymbol{h}_i^2$ for each node i, where $\boldsymbol{W}^3 \in \mathbb{R}^{D\times D}$ is the output projection matrix.

We adopt the loss function in [40], which preserves both local and global network structures, to train this GCN model.

Then based on the above initial graph embedding, the cascaded Transformer policy network is unrolled for T transition steps to simultaneously update the solution and the graph embeddings. Together, these two components constitute the proposed L2A model introduced in Sec. III.

Baseline comparison. We compare the proposed L2A scheme with the following **population-based** baseline methods that are also trained across graph distributions:

- **S2V-DQN** [22]: A greedy reinforcement-learning baseline that combines updated graph embeddings with fitted Q-learning and incrementally constructs the solution one node at a time.
- **ECO-DQN** [5]: An exploration-based reinforcement-learning baseline for Max-Cut, which allows reversible search moves and performs test-time exploration to iteratively improve the current solution.
- **PI-GNN** [34]: An unsupervised physics-inspired graph neural network baseline, which optimizes a relaxed QUBO/Ising-type energy and then projects the continuous output to a discrete solution.
- **iSCO** [38]: An iterative sampling baseline that improves the solution from scratch at test time without prior training.

To faithfully reproduce the neural baselines, we directly use the pretrained models released in their open-source implementations for "S2V-DQN", "ECO-DQN", and "PI-GNN" [2]. Notably, those GNN baselines are trained on smaller graphs with up to 200-300 nodes and then tested on larger graphs with up to 1000-1200 nodes. In contrast, our proposed model is trained and tested on graphs whose node numbers vary from 100, 200 to 1200. To ensure a fair comparison, during training of the proposed model, fresh graph instances are sampled using the same **graph generation functions** as those adopted in the baseline works; during inference, all methods are evaluated on the same test set, which contains 30 instances for each graph size in the set $\{100, 200, \ldots, 1100, 1200\}$.

Performance metric. For each graph instance in the test set, we first run the Gurobi solver within one hour and record the best result as the reference solution. All approximation ratios reported in this paper are measured with respect to the corresponding Gurobi solution, i.e.,

$$\eta = \frac{f(\boldsymbol{x}, G) - f(\overline{\boldsymbol{x}}, G)}{f(\overline{\boldsymbol{x}}, G)}, \tag{63}$$

where $\overline{\boldsymbol{x}}$ denotes the best (possibly optimal) solution produced by the Gurobi solver within one hour. In addition, we report the inference time, defined as the wall-clock time required to produce one test solution.

B. Ablation Study

This subsection presents ablation studies on the key components of the proposed L2A-for-GCO framework in Sec. III-Sec. V, including the Transformer architecture, the filter function as the CL method, the random masking as the CL method, and the blockwise sampling method.

We first examine the effectiveness of applying the **random masking** on graph nodes, which acts as a CL training scheme. The model is trained by using BA graph samples with the total node number $N = 100$ and the CL schedule $[N_1, N_2, N_3] = [80, 90, 100]$ (i.e., $L = 3$). At each training stage ℓ, we randomly mask out $N - N_\ell$ nodes on each graph sample. The solution vector at each transition step is generated based on the blockwise auto-regressive sampling, with block/grouping size 10 nodes. For comparison, we consider the following baselines: (1) The model is trained on BA graph samples with N_ℓ graph nodes throughout the entire training process, for $\ell \in [L]$. This yields L baselines; and (2) the Gurobi solver that runs within one hour and records the best solution. Fig. 7 plots the convergence curves of the loss in (9) versus training epochs for the proposed scheme and all baselines, while Fig. 8 plots the on-the-fly testing curves of the Max-cut score versus training epochs. Fig. 7 shows that the training loss gradually decreases and finally converges for each learning scheme. In addition, it is seen in Fig. 8 that the proposed scheme outperforms all other learning-based baselines while is only slightly inferior to the Gurobi benchmark, and its convergence curve exhibits a stepwise pattern, indicating that the proposed CL training strategy is effective. Furthermore, Fig. 9 displays the test performance over a pre-determined inference set versus transition step $t \in [T]$, based on the well-trained model. It is observed that the average testing score monotonically increases as t increases, justifying the effectiveness of the proposed recursive search process.

We then provide a comparison between different **sampling methods**, including the mean-field, the auto-regressive, and the blockwise auto-regressive sampling methods, under BA

[2]S2V-DQN: https://github.com/Hanjun-Dai/graph_comb_opt; ECO-DQN: https://github.com/tomdbar/eco-dqn; PI-GNN: https://github.com/amazon-science/co-with-gnns-example.

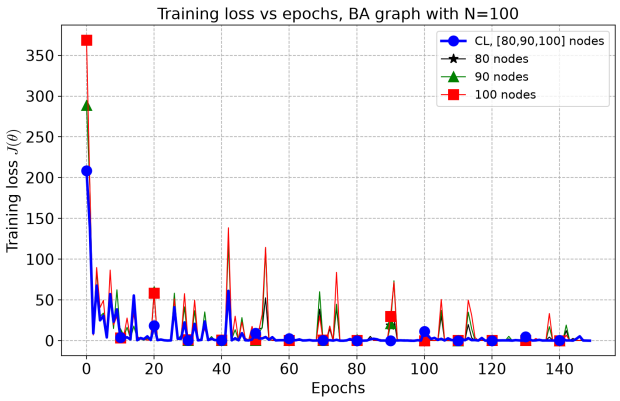


Fig. 7. Training loss curves of the proposed CL scheme and other baselines.

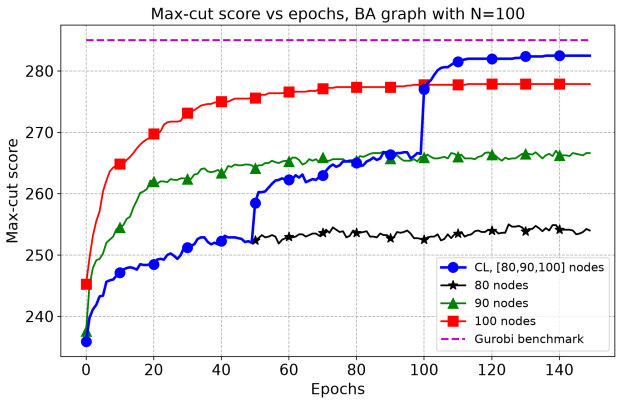


Fig. 8. On-the-fly testing curves of the proposed CL scheme and other baselines.

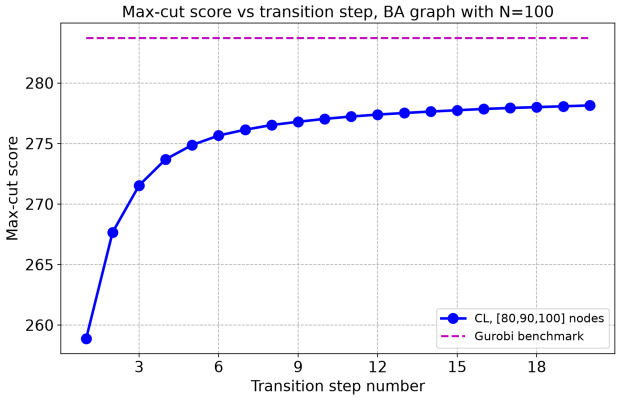


Fig. 9. Testing performance under different transition steps.

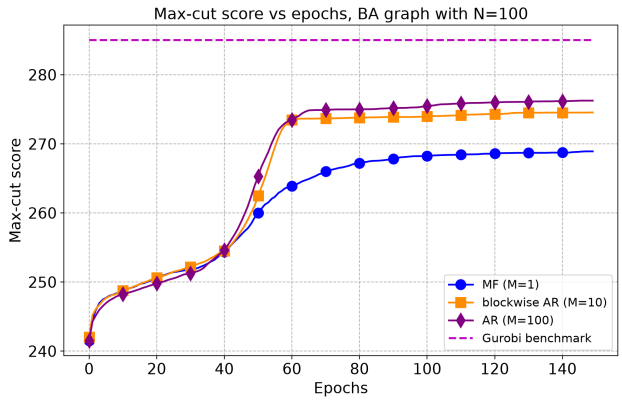


Fig. 10. On-the-fly testing curves under different sampling methods.

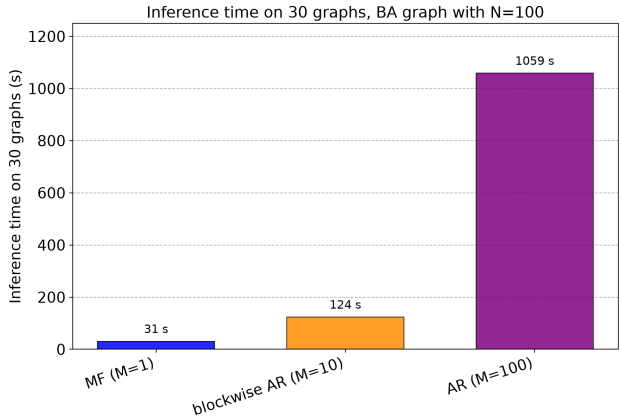


Fig. 11. Average inference times under different sampling methods.

graph instances with $N = 100$. Fig. 10 and Fig. 11 plot the on-the-fly testing curves and the average inference times under all sampling methods. It shows that in terms of the convergence performance, the blockwise sampling method significantly outperforms the mean-field counterpart, since the conditional forward process provides a superior dependency modeling among all graph nodes; the auto-regressive sampling slightly exceeds the blockwise sampling, due to a finer-grained dependency modeling via bit-by-bit generation. On the other hand, the execution time of the blockwise sampling is much shorter than the costly auto-regressive sampling, and is a bit longer than the fully-parallel mean-field sampling method. Consequently, the blockwise sampling method achieves an excellent balance between the performance and the inference overhead, and hence is chosen as the default sampling method in the subsequent experiments.

Next, we demonstrate the improvement brought by the **Transformer architecture**. Specifically, we compare the proposed Transformer-based policy network with: (1) the fully-connected policy network, and (2) the RNN-based policy network, under BA graph samples with $N = 300$. Fig. 12 and Fig. 13 plot the on-the-fly convergence curves and the inference performance over a pre-determined set under all network architectures. It shows that the Transformer architecture exhibits a superior learning performance over the other networks, and this advantage becomes more evident under larger-scale graphs due to the excellent scalability of the Transformer network.

Finally, we conduct an ablation study on the **filter function-enhanced CL training** scheme, where both the step number m in the filter function $S_m(\cdot)$ and the annealing parameter λ are gradually decreased. For comparison, we consider three baselines: (1) a scheme without filter functions and without annealing, (2) a scheme without filter functions but with the annealing schedule on λ, and (3) a scheme with a fixed filter

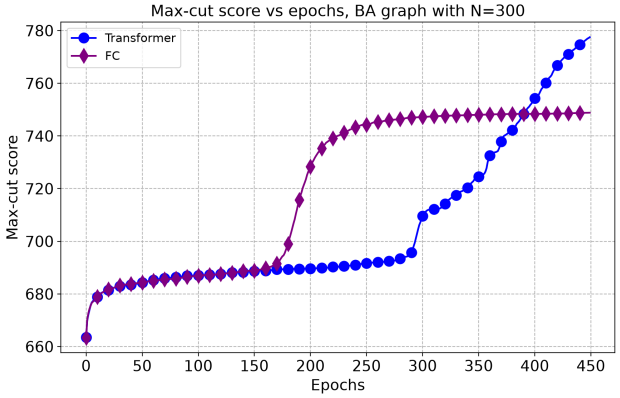


Fig. 12. On-the-fly testing curves under different network architectures.

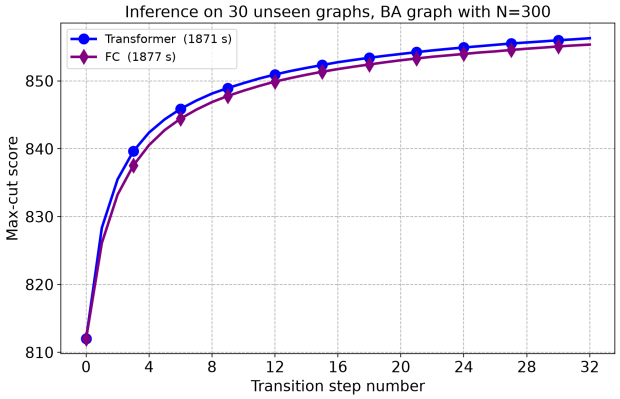


Fig. 13. Testing performance under different networks and transition steps.

function $S_m(\cdot)$ and the same annealing schedule on λ, where m can be fixed at each value in the curriculum. Fig. 12 and Fig. 13 plot the on-the-fly convergence curves and the inference performance over a pre-determined set under all settings of parameters (λ, m). It shows that the proposed filter function-enhanced CL scheme outperforms the other baselines.

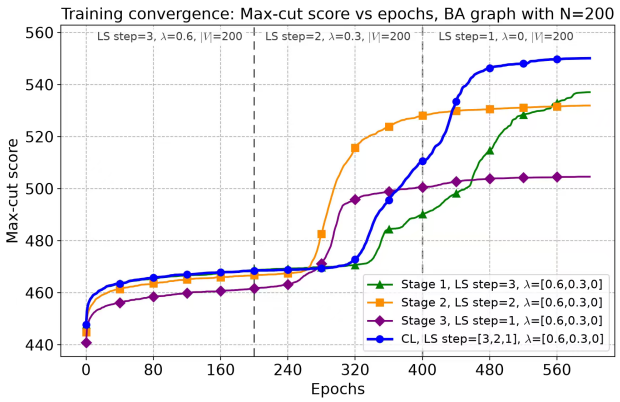


Fig. 14. On-the-fly testing curves under the proposed filter function-enhanced curriculum learning scheme and several baselines with different settings of (λ, m).

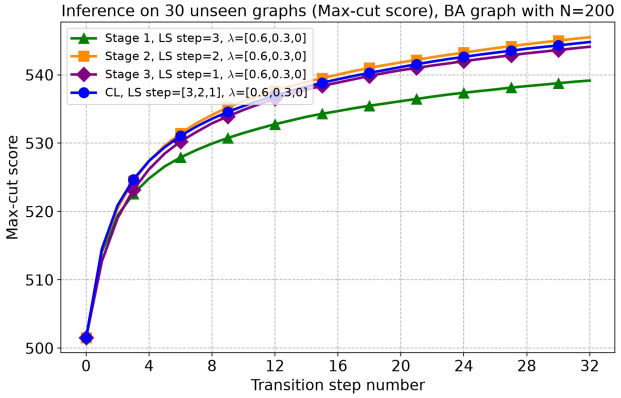


Fig. 15. Testing performance under different settings of (λ, m) and different transition steps.

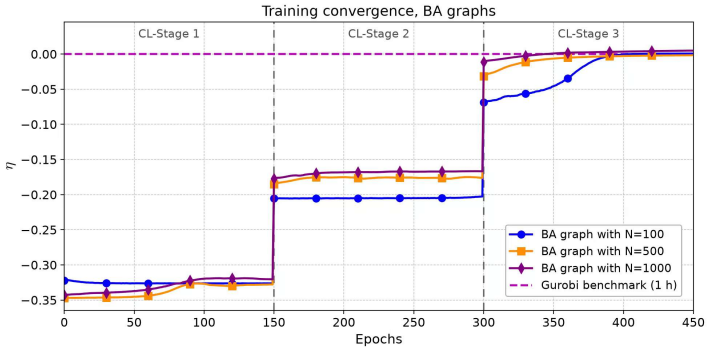


Fig. 16. Training convergence curves of the proposed schemes under different graph sizes.

C. Overall Performance

We now evaluate the overall performance of the proposed GCO L2A framework. Specifically, the proposed scheme adopts: (1) the random masking on graph nodes during training, (2) the blockwise auto-regressive sampling method, (3) the policy-gradient training strategy, (4) the GCO-Transformer as the policy network, and (5) the proposed filter function-enhanced CL scheme. Unless otherwise specified, the above settings are adopted in the proposed scheme throughout the following evaluations.

We first examine the **training convergence** of the proposed scheme on larger graph samples. As shown in Fig. 16, the proposed model still exhibits stable convergence when the graph size increases. The training curve decreases smoothly and reaches a favorable final objective value, indicating that the overall framework remains trainable under a larger-scale setting. This result is consistent with the observations in the ablation study and further confirms that the combination of filtering, curriculum learning, actor-critic training, and the GCO-Transformer leads to a robust training process.

We then compare the proposed method with other **distribution-level baselines**, including S2V-DQN, ECO-DQN, PI-GNN, and X2GNN. All schemes are trained and tested on the same graph family and evaluated on the same testing batches. As shown in Table III, the proposed method consistently achieves the best or near-best testing performance over

TABLE III
OVERALL PERFORMANCE COMPARISON ON THE BA GRAPH DISTRIBUTION. EACH RESULT IS AVERAGED OVER 30 INDEPENDENTLY GENERATED TEST GRAPHS. HIGHER VALUES INDICATE BETTER SOLUTION QUALITY.

Graph Size	S2V-DQN [22]	ECO-DQN [5]	X2GNN [1]	Gurobi	GCO-L2A
100	281.00	**283.70**	283.67	**283.70**	**283.70**
200	571.47	582.50	583.20	**583.27**	**583.27**
300	852.23	875.73	880.30	**880.43**	**880.43**
400	1135.37	1168.83	1179.47	1179.33	**1179.73**
500	1422.43	1461.37	1479.50	1475.67	**1479.73**
600	1682.50	1752.60	1779.73	1772.83	**1780.13**
700	1932.73	2041.43	2076.57	2067.50	**2077.43**
800	2182.10	2331.30	2376.87	2361.57	**2378.17**
900	2425.47	2616.93	2674.17	2656.93	**2675.53**
1000	2706.17	2911.93	2973.87	2951.23	**2975.43**
1100	2935.53	3180.77	3272.20	3250.17	**3273.93**
1200	3206.43	3476.03	3572.27	3547.07	**3574.00**

TABLE IV
OVERALL PERFORMANCE COMPARISON ON THE ER GRAPH DISTRIBUTION. EACH RESULT IS AVERAGED OVER 30 INDEPENDENTLY GENERATED TEST GRAPHS. HIGHER VALUES INDICATE BETTER SOLUTION QUALITY. THE BEST RESULT IN EACH ROW IS SHOWN IN BOLD.

Graph Size	S2V-DQN [22]	ECO-DQN [5]	X2GNN [1]	Gurobi	GCO-L2A
100	466.27	500.07	**500.33**	**500.33**	**500.33**
200	1785.67	1869.47	**1872.43**	1871.33	**1872.43**
300	3929.33	4056.93	4067.50	4066.87	**4067.63**
400	6891.37	7050.03	7074.60	7075.70	**7075.80**
500	10656.80	10843.43	10890.77	10893.03	**10893.93**
600	15196.07	15408.73	15473.63	15479.33	**15480.33**
700	20516.23	20774.67	20866.23	20876.27	**20877.93**
800	26633.27	26919.83	27042.47	27055.17	**27059.07**
900	33524.17	33855.00	34011.97	34032.07	**34035.67**
1000	41181.30	41575.00	41765.97	41796.13	**41799.90**
1100	49594.57	50073.37	50277.80	50311.30	**50316.27**
1200	58833.30	59342.83	59591.00	59627.53	**59634.60**

different graph sizes. Compared with the competing learning-based methods, the proposed scheme benefits from the semi-amortized L2A design, the filtering-based curriculum, and the Transformer policy architecture, and therefore produces higher-quality solutions in a more stable manner. These results verify that the proposed framework provides a clear performance advantage among population-based solvers.

Next, we report the **runtime comparison** with the same set of distribution-level baselines. The average inference time is presented in Fig 17, and the training time may also be included when needed. The results show that the proposed method achieves a favorable tradeoff between solution quality and computational cost. Although the proposed framework includes additional components such as filtering and multi-step policy unrolling, its inference remains efficient due to the amortized nature of the learned solver. Therefore, the proposed method is able to deliver strong overall performance in terms of both testing quality and practical runtime.

Finally, we evaluate the **generalizability** of the trained Transformer model across different graph sizes. Existing learning-based GCO methods are often trained on small graph instances and then tested on larger ones. In contrast, the proposed GCO-Transformer provides better scalability and enables training directly on larger graph samples. Moreover, the random-masking strategy used during training exposes the policy network to different active-node subsets, which naturally supports inference on smaller graph instances.

Let N_{t} denote the number of nodes in the training graphs and N_{i} denote the number of nodes in the testing graphs. We train the proposed model on graph instances with size N_{t} and evaluate the same pretrained model on test instances with different values of N_{i}. No additional fine-tuning is performed during testing. When $N_{\mathrm{i}} < N_{\mathrm{t}}$, the inactive nodes are masked out in the attention computation and solution sampling

TABLE V
GENERALIZATION PERFORMANCE OF GCO-L2A ON MAXCUT PROBLEMS IN BA GRAPHS. EACH RESULT (%) IS AVERAGED OVER 30 INDEPENDENTLY GENERATED TEST GRAPHS. HIGHER VALUES INDICATE BETTER SOLUTION QUALITY.

Test →	**BA graphs**											
Train ↓	$\|V\| = 100$	$\|V\| = 200$	$\|V\| = 300$	$\|V\| = 400$	$\|V\| = 500$	$\|V\| = 600$	$\|V\| = 700$	$\|V\| = 800$	$\|V\| = 900$	$\|V\| = 1000$	$\|V\| = 1100$	$\|V\| = 1200$
$\|V\| = 100$	0	–	–	–	–	–	–	–	–	–	–	–
$\|V\| = 200$	0	0	–	–	–	–	–	–	–	–	–	–
$\|V\| = 300$	0	0	-0.0189	–	–	–	–	–	–	–	–	–
$\|V\| = 400$	0	0	-0.0189	0	–	–	–	–	–	–	–	–
$\|V\| = 500$	0	0	-0.0038	-0.0339	0.1762	–	–	–	–	–	–	–
$\|V\| = 600$	0	0	-0.0076	-0.0141	0.0678	0.0696	–	–	–	–	–	–
$\|V\| = 700$	0	0	-0.0038	0.0057	0.1852	0.2670	0.3305	–	–	–	–	–
$\|V\| = 800$	0	0	-0.0151	0.0085	0.1762	0.2294	0.1870	0.4277	–	–	–	–
$\|V\| = 900$	0	-0.0057	-0.0151	0.0028	0.2169	0.2538	0.3096	0.4728	0.5671	–	–	–
$\|V\| = 1000$	0	-0.0057	-0.0038	-0.0933	0.2214	0.2877	0.2805	0.5095	0.4855	0.4653	–	–
$\|V\| = 1100$	0	0	-0.0114	-0.0141	0.1897	0.2802	0.2644	0.4644	0.5658	0.5726	0.4769	–
$\|V\| = 1200$	0	0	-0.0151	0.0028	0.1965	0.2294	0.2773	0.4446	0.5181	0.5568	0.4164	0.4285

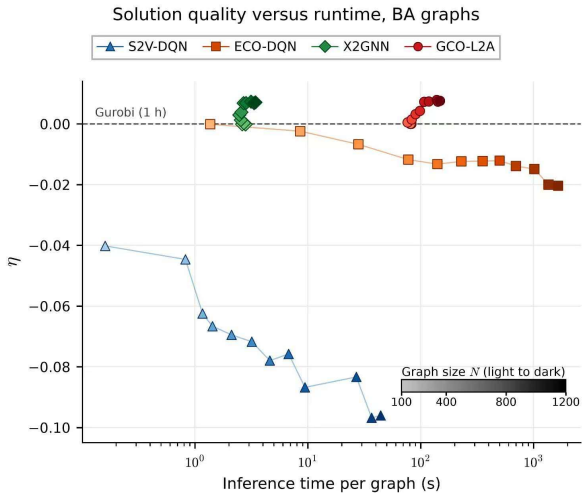


Fig. 17. Performance ratio versus runtime per graph sample under different population-based schemes.

process. The results are reported in Table. V. The proposed model maintains strong approximation ratios when transferred from the training size to smaller testing sizes, showing that random masking effectively improves size generalization. In particular, the performance degradation is marginal for in-distribution graph families, which indicates that the learned GCO-Transformer policy does not simply memorize a fixed-dimensional solution pattern. Instead, it learns a distribution-level transition rule that can be applied to different active graph scales. These results verify the generalizability of the proposed population-based L2A framework and support its use as a scalable pretrained solver for graph instances of varying sizes.

VII. CONCLUSION

In this paper, we proposed a population-based L2A framework for GCO problems. The framework learns a stochastic transition policy over graph distributions and recursively refines candidate solutions, thereby shifting part of the computational burden from online optimization to offline training. To instantiate this policy, we developed a random-masking sampling method that factorizes the transition distribution through conditional node unmasking and unifies mean-field and autoregressive sampling as two special cases. Built on this scheme, the extended Transformer network jointly processes the current solution and graph embedding, enabling conditional solution sampling, global node correlations, and iterative solution-feature updates. We further introduced curriculum learning strategies to stabilize training on large-scale GCO problems with irregular objective landscapes. The filter-function-based curriculum smooths the effective objective and gradually approaches the original objective, while the attention-masking curriculum progressively expands the active graph scale during Transformer training. We also developed policy-gradient and actor-critic schemes for trajectory-level learning. Experiments on Max-Cut over multiple graph distributions show that L2A achieves high-quality solutions with low inference latency after pretraining. The proposed method outperforms strong learning-based baselines, is competitive with or better than time-limited Gurobi in several large-scale settings, and generalizes well across graph sizes.

APPENDIX
PROOF OF (14),(15)

According to (15), $b(G)$ is independent of τ. By exchanging the order of differentiation and summation, the following equality holds:

$$\begin{aligned}
&\mathbb{E}_{G\sim\mathbb{D}}\ \mathbb{E}_{\tau\sim\Pi_\theta}\left[b(G)\ \nabla_\theta \log p(\tau|\theta,\boldsymbol{Z}(G))\right]\\
&= \mathbb{E}_{G\sim\mathbb{D}}\left[b(G)\ \sum_\tau p(\tau|\theta,\boldsymbol{Z}(G))\nabla_\theta \log p(\tau|\theta,\boldsymbol{Z}(G))\right]\\
&= \mathbb{E}_{G\sim\mathbb{D}}\left[b(G)\ \sum_\tau \nabla_\theta p(\tau|\theta,\boldsymbol{Z}(G))\right]\\
&= \mathbb{E}_{G\sim\mathbb{D}}\left[b(G)\ \nabla_\theta \underbrace{\sum_\tau p(\tau|\theta,\boldsymbol{Z}(G))}_{=1}\right] = 0. \qquad (64)
\end{aligned}$$

where the second equality uses

$$p(\tau|\theta,\boldsymbol{Z}(G))\nabla_\theta \log p(\tau|\theta,\boldsymbol{Z}(G)) = \nabla_\theta p(\tau|\theta,\boldsymbol{Z}(G)). \quad (65)$$

The gradient of the log probability of trajectory τ in (13) is given by

$$\begin{aligned}
&\nabla_\theta \log p(\tau|\theta, \boldsymbol{Z}(G)) \\
&= \nabla_\theta \log \left[p(\boldsymbol{x}^0) \prod_{t=1}^{T} \Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1}, \boldsymbol{Z}(G)) \right] \\
&= \nabla_\theta \log p(\boldsymbol{x}^0) + \sum_{t=1}^{T} \nabla_\theta \log \Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1}, \boldsymbol{Z}(G)) \\
&= \sum_{t=1}^{T} \nabla_\theta \log \Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1}, \boldsymbol{Z}(G)). \qquad (66)
\end{aligned}$$

We take the derivative of $J_\lambda(\theta)$ in (11) with respect to θ and obtain

$$\begin{aligned}
&\nabla_\theta J_\lambda(\theta) \\
&= \nabla_\theta \mathbb{E}_{G\sim\mathbb{D}} \mathbb{E}_{\tau=(\boldsymbol{x}^0,\boldsymbol{x}^1,\cdots,\boldsymbol{x}^T):\boldsymbol{x}^t\sim\Pi_\theta(\cdot|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G))} \left[J_\lambda(\theta;\tau,G) \right] \\
&= \nabla_\theta \mathbb{E}_{G\sim\mathbb{D}} \int_\tau \left(g(\tau,G) + \lambda \log p(\tau|\theta,\boldsymbol{Z}(G)) \right) p(\tau|\theta,\boldsymbol{Z}(G)) d\tau \\
&= \mathbb{E}_{G\sim\mathbb{D}} \left[\lambda \int_\tau p(\tau|\theta,G) \nabla_\theta \log p(\tau|\theta,G) \right. \\
&\left. + \int_\tau \left(g(\tau,G) + \lambda \log p(\tau|\theta,\boldsymbol{Z}(G)) \right) p(\tau|\theta,G) \nabla_\theta \log p(\tau|\theta,G) \right] \\
&= \mathbb{E}_{G\sim\mathbb{D}} \left[\int_\tau \left(g(\tau,G) + \lambda \log p(\tau|\theta,\boldsymbol{Z}(G)) \right) \right. \\
&\qquad \left. p(\tau|\theta,G) \nabla_\theta \log p(\tau|\theta,G) \right] \\
&= \mathbb{E}_{G\sim\mathbb{D}} \, \mathbb{E}_{\tau\sim\Pi_\theta} \left[\left(g(\tau,G) + \lambda \log p(\tau|\theta,\boldsymbol{Z}(G)) \right) \right. \\
&\qquad \left. \left(\sum_{t=1}^{T} \nabla_\theta \log \Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G)) \right) \right] \qquad (67)
\end{aligned}$$

where the third equality used the trick of log-probability of a trajectory such that $\nabla_\theta p(\tau|\theta,\boldsymbol{Z}(G)) = p(\tau|\theta,\boldsymbol{Z}(G)) \nabla_\theta \log p(\tau|\theta,\boldsymbol{Z}(G))$, the fourth equality used (64), and the last equality used (66).

Adding (64) into (67) yields the results in (14).

APPENDIX
PROOF OF (18)-(20)

Using the log-derivative trick in (64), We take the derivative of $J_\lambda(\theta)$ in (11) with respect to θ and obtain

$$\begin{aligned}
&\nabla_\theta J_\lambda(\theta) \\
&= \nabla_\theta \mathbb{E}_{G\sim\mathbb{D}} \mathbb{E}_{\tau=(\boldsymbol{x}^0,\boldsymbol{x}^1,\cdots,\boldsymbol{x}^T):\boldsymbol{x}^t\sim\Pi_\theta(\cdot|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G))} \left[J_\lambda(\theta;\tau,G) \right] \\
&= \mathbb{E}_{G\sim\mathbb{D}} \int_\tau L_\theta(\tau,G,\lambda) p(\tau|\theta,\boldsymbol{Z}(G)) \cdot \nabla_\theta \log p(\tau|\theta,\boldsymbol{Z}(G)) d\tau \\
&= \mathbb{E}_{G\sim\mathbb{D}} \int_\tau L_\theta(\tau,G,\lambda) p(\tau|\theta,\boldsymbol{Z}(G)) \cdot \\
&\qquad \sum_{t=1}^{T} \nabla_\theta \log \Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G)) d\tau. \qquad (68)
\end{aligned}$$

By (16), the value function is equivalently:

$$V(\boldsymbol{x}^t, G) = \mathbb{E}\left[R(\tau^t,G,\lambda) \mid \boldsymbol{x}^t, G \right]. \qquad (69)$$

Moreover, for each t, we have

$$\begin{aligned}
&\mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)} \left[\sum_{t=1}^{T} V(\boldsymbol{x}^{t-1},G) \nabla_\theta \log \Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G)) \right] \\
&= \sum_{t=1}^{T} \mathbb{E}_{\boldsymbol{x}^t\sim\Pi_\theta(\cdot|\boldsymbol{x}^{t-1})} \left[V(\boldsymbol{x}^{t-1},G) \nabla_\theta \log \Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G)) \right] \\
&= \sum_{t=1}^{T} V(\boldsymbol{x}^{t-1},G) \nabla_\theta \left[\sum_{\boldsymbol{x}^t} \Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G)) \right] = 0, \qquad (70)
\end{aligned}$$

and hence subtracting this term does not change the expectation. Therefore, the derivative in (68) can be rewritten as follows:

$$\begin{aligned}
&\nabla_\theta J_\lambda(\theta) \\
&= \mathbb{E}_{G\sim\mathbb{D}} \mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)} \left[\sum_{t=1}^{T} R(\tau^{t-1},G,\lambda) \cdot \right. \\
&\qquad \left. \nabla_\theta \log \Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G)) \right] \\
&= \mathbb{E}_{G\sim\mathbb{D}} \mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)} \left[\sum_{t=1}^{T} \left(R(\tau^{t-1},G,\lambda) - V(\boldsymbol{x}^{t-1},G) \right) \cdot \right. \\
&\qquad \left. \nabla_\theta \log \Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G)) \right]. \qquad (71)
\end{aligned}$$

Similar to the policy gradient case in (14), we have the following gradient results in the actor-critic cases:

$$\begin{aligned}
\nabla_\theta J_\lambda(\theta) = \mathbb{E}_{G\sim\mathbb{D}} \mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)} &\left[\sum_{t=1}^{T} A_{\lambda,t-1} \cdot \right. \\
&\left. \nabla_\theta \log \Pi_\theta(\boldsymbol{x}^t|\boldsymbol{x}^{t-1},\boldsymbol{Z}(G)) \right], \qquad (72)
\end{aligned}$$

where for $t = 0, \ldots, T-1$, the step-wise advantage function is

$$A_{\lambda,t} \triangleq R(\tau^t,G,\lambda) - V(\boldsymbol{x}^t,G). \qquad (73)$$

Then we update the critic model as follows. Note that the critic should approximate the value function, i.e.,

$$V_{\boldsymbol{\phi}}(\boldsymbol{x}^t,G) \approx V(\boldsymbol{x}^t,G) = \mathbb{E}[R(\tau^t,G,\lambda) \mid \boldsymbol{x}^t, G], \qquad (74)$$

hence the objective of the critic $J_V(\boldsymbol{\phi},\boldsymbol{\theta})$ becomes

$$\min_{\boldsymbol{\phi}} \; \frac{1}{2} \cdot \mathbb{E}_{G\sim\mathbb{D}} \mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)} \left[\sum_{t=0}^{T-1} (V_{\boldsymbol{\phi}}(\boldsymbol{x}^t,G) - R(\tau^t,G,\lambda))^2 \right], \qquad (75)$$

where the critic errors over all steps are considered. And the corresponding gradient is as follows

$$\begin{aligned}
&\nabla_{\boldsymbol{\phi}} J_V(\boldsymbol{\phi},\boldsymbol{\theta}) = \mathbb{E}_{G\sim\mathbb{D}} \mathbb{E}_{\tau\sim\Pi_\theta(\cdot|G)} \\
&\left[\sum_{t=0}^{T-1} \left(V_{\boldsymbol{\phi}}(\boldsymbol{x}^t,G) - R(\tau^t,G,\lambda) \right) \cdot \nabla_{\boldsymbol{\phi}} V_{\boldsymbol{\phi}}(\boldsymbol{x}^t,G) \right], \qquad (76)
\end{aligned}$$

where the term $|(V_{\boldsymbol{\phi}}(\boldsymbol{x}^t,G) - R(\tau^t,G,\lambda)|$ is typically referred to as the temporal-difference (TD) error at the tth step.

REFERENCES

[1] Utku Umur ACIKALIN, Aaron M Ferber, and Carla P Gomes. Learning to explore and exploit with GNNs for unsupervised combinatorial optimization. In *The Thirteenth International Conference on Learning Representations*, 2025.

[2] Zafarali Ahmed, Nicolas Le Roux, Mohammad Norouzi, and Dale Schuurmans. Understanding the impact of entropy on policy optimization. In *Proceedings of the 36th International Conference on Machine Learning*, pages 151–160, 2019.

[3] Brandon Amos. Tutorial on amortized optimization. *Foundations and Trends in Machine Learning*, 16(5):592–732, 2023.

[4] Francisco Barahona. On the computational complexity of ising spin glass models. *Journal of Physics A: Mathematical and General*, 15(10):3241–3253, 1982.

[5] Thomas Barrett, William Clements, Jakob Foerster, and Alex Lvovsky. Exploratory combinatorial optimization with reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 3243–3250, 2020.

[6] Irwan Bello, Hieu Pham, Quoc V. Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. In *ICLR Workshop*, 2017.

[7] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning*, pages 41–48, 2009.

[8] Hans L. Bodlaender and Klaus Jansen. On the complexity of the maximum cut problem. *Nordic Journal of Computing*, 7(1):14–31, 2000.

[9] Michael J. Brusco and Stephanie Stahl. *Branch-and-Bound Applications in Combinatorial Data Analysis*. Springer, 2005.

[10] Cheng Chen, Ruitao Chen, Tianyou Li, Ruichen Ao, and Zaiwen Wen. Monte carlo policy gradient method for binary optimization. *arXiv preprint arXiv:2307.00783*, 2023.

[11] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[12] Kathryn Anne Dowsland and Jonathan Thompson. Simulated annealing. In *Handbook of Natural Computing*, pages 1623–1655. Springer, 2012.

[13] Darko Drakulic, Sofia Michel, Florian Mai, Arnaud Sors, and Jean-Marc Andreoli. BQ-NCO: Bisimulation quotienting for efficient neural combinatorial optimization. In *Advances in Neural Information Processing Systems*, volume 36, 2024.

[14] James Durbin and Siem Jan Koopman. *Time Series Analysis by State Space Methods*. Oxford University Press, 2012.

[15] Abraham D. Flaxman, Adam Tauman Kalai, and H. Brendan McMahan. Online convex optimization in the bandit setting: Gradient descent without a gradient. *arXiv preprint cs/0408007*, 2004.

[16] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. Pearson, 2009.

[17] Evan Greensmith, Peter L. Bartlett, and Jonathan Baxter. Variance reduction techniques for gradient estimates in reinforcement learning. *Journal of Machine Learning Research*, 5:1471–1530, 2004.

[18] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022.

[19] Mohamed Hibat-Allah, Estelle M. Inack, Roeland Wiersema, Roger G. Melko, and Juan Carrasquilla. Variational neural annealing. *Nature Machine Intelligence*, 3(11):952–961, 2021.

[20] Jeremy Johnston, Xiao-Yang Liu, Shixun Wu, and Xiaodong Wang. A curriculum learning approach to optimization with application to downlink beamforming. *IEEE Transactions on Signal Processing*, 72:84–98, 2023.

[21] Nikolaos Karalias and Andreas Loukas. Erdos goes neural: An unsupervised learning framework for combinatorial optimization on graphs. In *Advances in Neural Information Processing Systems*, volume 33, pages 6659–6672, 2020.

[22] Elias B. Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. Learning combinatorial optimization algorithms over graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[23] Scott Kirkpatrick, C. Daniel Gelatt Jr., and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[24] Jon Kleinberg and Éva Tardos. *Algorithm Design*. Pearson, 2006.

[25] Gary A. Kochenberger, Jin-Kao Hao, Zhipeng L"u, Haibo Wang, and Fred Glover. Solving large scale max cut problems via tabu search. *Journal of Heuristics*, 19:565–571, 2013.

[26] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations*, 2019.

[27] Yang Li, Jinpei Guo, Runzhong Wang, and Junchi Yan. T2T: From distribution learning in training to gradient search in testing for combinatorial optimization. In *Advances in Neural Information Processing Systems*, volume 36, pages 50020–50040, 2023.

[28] Andrew Lucas. Ising formulations of many NP problems. *Frontiers in Physics*, 2:5, 2014.

[29] Fu Luo, Xi Lin, Fei Liu, Qingfu Zhang, and Zhenkun Wang. Neural combinatorial optimization with heavy decoder: Toward large scale generalization. In *Advances in Neural Information Processing Systems*, volume 36, pages 8845–8864, 2023.

[30] Hugues Marchand, Alexander Martin, Robert Weismantel, and Laurence Wolsey. Cutting planes in integer and mixed integer programming. *Discrete Applied Mathematics*, 123(1–3):397–446, 2002.

[31] Yimeng Min, Yiwei Bai, and Carla P. Gomes. Unsupervised learning for solving the travelling salesman problem. In *Advances in Neural Information Processing Systems*, volume 36, 2024.

[32] Christos H. Papadimitriou. The euclidean traveling salesman problem is NP-complete. *Theoretical Computer Science*, 4(3):237–244, 1977.

[33] Sebastian Sanokowski, Wilhelm Berghammer, Sepp Hochreiter, and Sebastian Lehner. Variational annealing on graphs for combinatorial optimization. In *Advances in Neural Information Processing Systems*, volume 36, 2024.

[34] Martin J. A. Schuetz, J. Kyle Brubaker, and Helmut G. Katzgraber. Combinatorial optimization with physics-inspired graph neural networks. *Nature Machine Intelligence*, 4(4):367–377, 2022.

[35] John Schulman, Nicolas Heess, Theophane Weber, and Pieter Abbeel. Gradient estimation using stochastic computation graphs. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[36] Predrag Spasojevic and Xiaodong Wang. Improved robust multiuser detection in non-gaussian channels. *IEEE Signal Processing Letters*, 8(3):83–86, 2001.

[37] Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[38] Haoran Sun, Katayoon Goshvadi, Azade Nova, Dale Schuurmans, and Hanjun Dai. Revisiting sampling for combinatorial optimization. In *Proceedings of the 40th International Conference on Machine Learning*, pages 32859–32874, 2023.

[39] Zhiqing Sun and Yiming Yang. DIFUSCO: Graph-based diffusion solvers for combinatorial optimization. In *Advances in Neural Information Processing Systems*, volume 36, pages 3706–3731, 2023.

[40] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. LINE: Large-scale information network embedding. In *Proceedings of the 24th International Conference on World Wide Web*, pages 1067–1077, 2015.

[41] Yunhao Tang, Shipra Agrawal, and Yuri Faenza. Reinforcement learning for integer programming: Learning to cut. In *Proceedings of the 37th International Conference on Machine Learning*, pages 9367–9376, 2020.

[42] Oriol Vinyals, Meire Fortunato, and Navdeep Jaitly. Pointer networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

[43] Nicholas Walker, Ka-Ming Tam, and Mark Jarrell. Deep learning on the two-dimensional ising model to extract the crossover region with a variational autoencoder. *Scientific Reports*, 10:13047, 2020.

[44] Haoyu Peter Wang, Nan Wu, Hang Yang, Cong Hao, and Pan Li. Unsupervised learning for combinatorial optimization with principled objective relaxation. In *Advances in Neural Information Processing Systems*, volume 35, pages 31444–31458, 2022.

[45] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3):229–256, 1992.

[46] Dinghuai Zhang, Hanjun Dai, Nikolay Malkin, Aaron C. Courville, Yoshua Bengio, and Ling Pan. Let the flows tell: Solving graph combinatorial optimization problems with GFlowNets. In *Advances in Neural Information Processing Systems*, volume 36, pages 11952–11969, 2023.

[47] Yubo Zhang, Xiao-Yang Liu, and Xiaodong Wang. Transformer-based scalable beamforming optimization via deep residual learning. *arXiv preprint arXiv:2510.13077*, 2025.

[48] Tingting Zhao, Gang Niu, Ning Xie, Jucheng Yang, and Masashi Sugiyama. Regularized policy gradients: Direct variance reduction in policy gradient estimation. In *Asian Conference on Machine Learning*, pages 333–348, 2016.